

# ANQI: An Approximate Reverse $k$ -NN Query Index at Scale

Anonymous Author(s)

## Abstract

Reverse  $k$ -nearest-neighbor (RkNN) search asks which base vectors would rank a query vector among their own  $k$  nearest neighbors. Unlike forward  $k$ -NN, RkNN membership depends jointly on the requested rank  $k$  and each base vector’s local neighborhood. Neighborhoods differ across base vectors, and the requested rank may differ across queries; supporting RkNN at scale therefore makes both query-time computation and threshold storage difficult.

We present ANQI to address both bottlenecks by separating candidate routing from threshold reconstruction. It lifts each object’s reverse cutoff into a Euclidean ANN score, allowing a standard graph to generate candidates instead of comparing the query with every object. A compact verifier then reconstructs the requested rank cutoff from a shared low-rank model and quantized residuals, avoiding a dense threshold table while retaining object-specific detail. For Euclidean thresholds, the Local Radius Law explains why this rank structure can be shared; for inner product, ANQI fits the same representation directly to signed thresholds. One graph and verifier therefore serve all materialized target ranks without rebuilding the index.

Across five million-scale datasets, ANQI delivers multi-fold throughput gains over recall-qualified baselines at high-recall operating points (recall 0.95 for  $L_2$  and 0.99 for IP). Precision is at least 0.99 at every reported operating point. Beyond these million-scale results, to our knowledge, ANQI is the first RkNN index to demonstrate an industrial-scale billion-vector construction-and-query result: on SIFT1B it achieves recall and precision above 0.95 at more than 10,000 QPS. The implementation is available at <https://github.com/Yujian-Fu/anqi-implementation>.

## PVLDB Reference Format:

Anonymous Author(s). ANQI: An Approximate Reverse  $k$ -NN Query Index at Scale. PVLDB, XX(X): XXX-XXX, 2027. doi:XX.XX/XXX.XX

## 1 Introduction

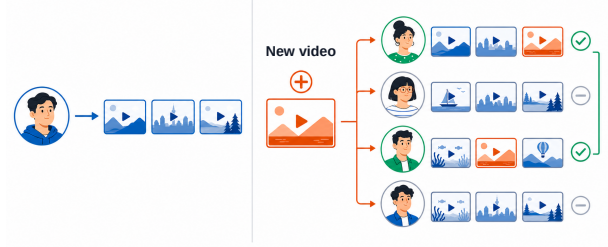
Conventional  $k$ -nearest-neighbor ( $k$ NN) search is query-centered: given a vector  $q$ , it returns the  $k$  base vectors closest to  $q$ . Reverse  $k$ -nearest-neighbor (RkNN) search asks whose own top- $k$  list would contain  $q$ . This direction supports item-driven retrieval. A seller can find the users most likely to rank a new or discounted item highly [3, 4]; a service can find subscribers for whom a new document is relevant, or customers affected by a new store location [26]. In the recommendation example in fig. 1, forward  $k$ NN selects videos for one user, whereas RkNN selects the initial audience for a new video.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. XX, No. X ISSN 2150-8097. doi:XX.XX/XXX.XX

Forward  $k$ NN

Reverse  $k$ NN



**Figure 1: Forward and reverse top- $k$  retrieval in a recommendation workload. Left: forward  $k$ NN retrieves the top- $k$  videos for one user. Right: after a new video  $q$  arrives, RkNN returns the users whose own top- $k$  lists would include  $q$ ; the green users are the returned set.**

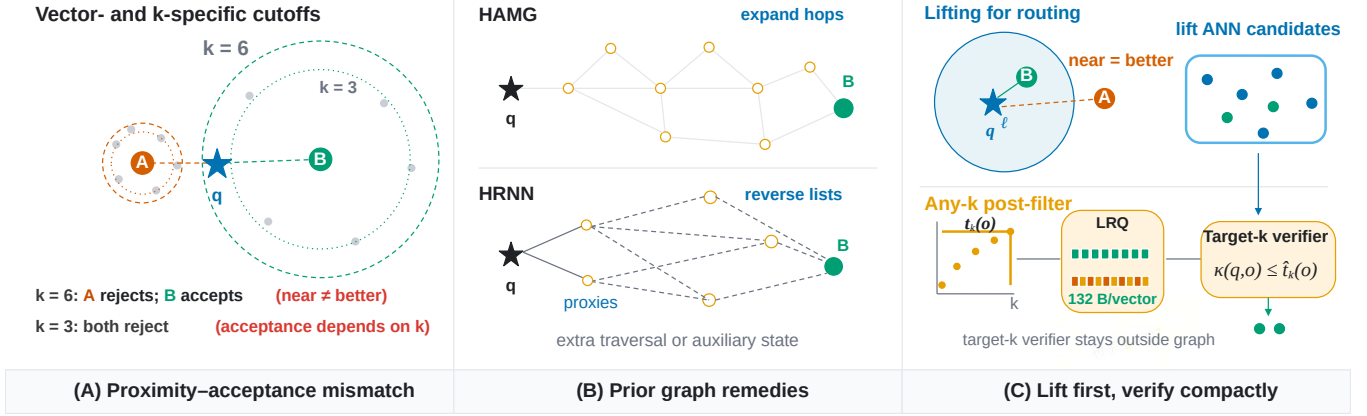
Modern embedding collections make the reverse query difficult for two reasons. First, forward proximity does not order reverse candidates. Each base vector judges  $q$  against its own neighborhood, so a nearby vector can reject  $q$  while a farther one accepts it. A proximity index can therefore miss true reverse neighbors even when it retrieves an accurate forward neighborhood. Second, the acceptance cutoff is specific to both the base vector and the requested  $k$ . Computing it during every query is expensive, while storing every cutoff creates a dense object-by-rank table. At million- and billion-vector scale, broader candidate exploration and larger verification state force a direct trade-off between retrieval quality, query efficiency, and memory.

High-dimensional RkNN methods use candidate generation to avoid a full scan and then verify the reverse predicate for each candidate. Figure 2(A) shows why both stages are difficult. For a base vector  $o$ , let  $r_k(o)$  be the distance to its  $k$ -th nearest other base vector. In  $L_2$  space,

$$o \in RkNN(q) \iff \delta(q, o)^2 \leq r_k(o)^2. \quad (1)$$

At  $k = 6$ , the nearby vector  $A$  lies in a dense region and rejects  $q$  because its acceptance radius is small. The farther vector  $B$  lies in a sparser region and accepts  $q$ . Base-space distance therefore ranks a rejector ahead of an acceptor. When the request changes to  $k = 3$ ,  $B$  also rejects  $q$ , illustrating that verification must recover a different cutoff for each rank. Existing systems close these gaps after ordinary routing. HAMG [36] expands multiple graph hops and HRNN [40] follows materialized reverse lists, increasing traversal or auxiliary state. SFT [35] and RDT [10] add range tests or forward- $k$ NN refinement. IP systems likewise scan bounded user blocks and may invoke item scans or hash searches for surviving users [3, 4, 24].

ANQI is a large-scale approximate RkNN index that changes candidate routing instead of repairing it afterward. Figure 2(C) shows the design. Proximity lifting embeds each base vector’s acceptance cutoff in its indexed score. A standard norm-equalization step then turns the reverse-aware score into Euclidean proximity, so mature ANN graphs such as HNSW [30], NSG [18], or DiskANN [37] can



**Figure 2: The two  $RkNN$  difficulties and ANQI’s response.** (A) For the blue-star query  $q$ , vector- and rank-specific cutoffs break the implication “nearer means more likely to accept.” At  $k = 6$ , nearby  $A$  rejects  $q$  while farther  $B$  accepts it. At  $k = 3$ , both reject it. (B) Representative graph systems repair candidate coverage after forward routing. HAMG [36] expands graph hops and HRNN [40] follows reverse lists from forward seeds. (C) ANQI encodes a reverse cutoff before routing. Forward ANN search then generates candidates in the lifted space and a compact target- $k$  verifier makes the final decision.

retrieve reverse candidates. The same graph-search substrate serves both  $L_2$  and IP after the corresponding score transformation. ANQI pairs this reverse-aware routing geometry with a compact target- $k$  verifier, separating candidate coverage from final membership decisions.

Candidate coverage and membership accuracy remain separate. The lifted graph finds a compact candidate set; an independent target- $k$  verifier makes the final decision. To avoid a dense threshold table, ANQI represents the object-by-rank matrix with a shared low-rank model and packed learned residual quantization (LRQ), as formalized in section 4.3. LRQ encodes the scalar residual left by the low-rank reconstruction, preserving object- and rank-specific boundary detail. Only the requested threshold column is decoded, and verifier state never enters graph distance. This division reduces both candidate-search work and persistent threshold storage.

Table 1 compares direct high-dimensional reverse-neighbor indexes. “Approx.” means that the query algorithm may return an inexact answer set. “Any- $k$ ” means that a query selects a materialized target rank from the range supported by one index without rebuilding its candidate graph. Scale is the largest collection evaluated in each cited work; ANQI’s 1B entry refers to the separate SIFT scaling prototype in section 6.4. ANQI uses one graph-and-verifier architecture for  $L_2$  and IP and extends the reported scale of the compared direct indexes from ten million to one billion vectors. Its main contributions are:

- We formulate high-dimensional  $RkNN$  membership under  $L_2$  and IP through a common threshold-MIPS interface. It encodes each base vector’s reverse decision boundary in the indexed score and enables one Euclidean candidate-search substrate for both predicates.
- We model vector- and rank-specific thresholds with a low-rank representation and LRQ. For  $L_2$ , the Local Radius Law provides a Euclidean rationale for shared rank structure; for IP, the same representation is fitted directly to signed thresholds. Residual codes recover boundary fidelity without a full-precision threshold table.

**Table 1: Capabilities and reported scale of high-dimensional reverse-neighbor indexes.**

| Method      | Index   | $L_2$ | IP | Any- $k$ | Approx. | Scale  |
|-------------|---------|-------|----|----------|---------|--------|
| SFT [35]    | Tree    | ✓     | –  | ✓        | ✓       | $10^4$ |
| RDT [10]    | Tree    | ✓     | –  | ✓        | ✓       | $10^6$ |
| HAMG [36]   | Graph   | ✓     | –  | ✓        | ✓       | $10^7$ |
| HRNN [40]   | Graph   | ✓     | –  | –        | ✓       | $10^6$ |
| SIMPFER [3] | Bounds  | –     | ✓  | –        | –       | $10^6$ |
| SAH [24]    | Hashing | –     | ✓  | –        | ✓       | $10^6$ |
| ANQI        | Graph   | ✓     | ✓  | ✓        | ✓       | $10^9$ |

- We evaluate the integrated system on five million-scale datasets under both predicates. ANQI achieves  $2.14\text{--}4.44\times$  the throughput of the fastest recall-qualified  $L_2$  baseline and  $10\text{--}2,900\times$  that of the fastest recall-qualified IP baseline at operating points selected for high recall (near 0.95 for  $L_2$  and 0.99 for IP). At these points, ANQI’s precision is at least 0.997 across the 15 reported dataset–target-rank configurations. An industrial-scale SIFT1B prototype demonstrates both construction and query processing, with both recall and precision above 0.95 at more than 10,000 QPS.

## 2 Related Work

ANQI sits at the intersection of proximity graph search, reverse-neighbor indexing, dimension lifting, and compact neighbor-distance modeling.

*Proximity graph indexes.* Proximity graphs accelerate forward nearest-neighbor search by routing a query through a sparse neighborhood graph instead of scanning the collection. NN-Descent [15] established scalable graph refinement, and EFANNA [17] combined tree seeding with graph search. HNSW [30] and NSG [18] developed navigable and pruned forward-ANN graphs. Vista [20] adapts graph connectivity to imbalanced data. DiskANN/Vamana [37] targets billion-scale search, while ParlayANN [32] provides parallel construction. Related work also studies quantized-search clusters and

billion-scale geometric partitions [19, 21]. Graph range retrieval returns a variable-cardinality set but starts from one query radius [31]. ANQI adopts a bounded-degree graph as its forward-search engine after lifting. It does so through a reverse-aware geometry that makes graph traversal useful for RkNN candidates while retaining mature forward-ANN construction and navigation.

*Reverse- $k$ NN indexes.* Reverse-neighbor search began as an influence query and later gained exact spatial and metric indexes [2, 26, 39]. Methods for high-dimensional data commonly separate candidate generation from vector-specific verification. SFT [35] starts from forward neighbors. Arthur–Oudot address reverse 1-NN through lifted LSH subproblems [5]. RDT [10] combines local tests with forward- $k$ NN refinement. Graph systems broaden candidate coverage in the base space. HAMG [36] uses multi-hop expansion, while HRNN [40] uses materialized reverse lists. HRNN is our closest direct competitor. Reverse-MIPS methods specialize the predicate to inner product. SIMPFER [3, 4] prunes candidates with per-user score bounds. Concurrent RT-RkNN [8] solves an exact two-dimensional spatial variant with GPU ray tracing. Its setting differs from high-dimensional vector search. ANQI retains the same two-stage decomposition. Its distinction is to incorporate reverse thresholds into candidate routing and to use one architecture for both  $L_2$  and inner product.

*Dimension lifting.* The  $L_2$  reverse predicate is a point-enclosure query over balls  $B(o, r_k(o))$ . Its signed power distance,  $\|q - o\|^2 - r_k(o)^2$ , belongs to the classical geometry of power diagrams and sphere-to-halfspace lifting [6]. Cheong et al. exploit related geometry for exact fixed-dimensional reverse-neighbor search, while Arthur–Oudot construct lifted LSH subproblems for high-dimensional reverse 1-NN [5, 14].

ANQI builds on this geometric lineage but targets a different high-dimensional index interface. It derives the scalar correction from a rank-dependent local-neighbor cutoff, builds one approximate graph for candidate generation, and applies compact target- $k$  verification. SAH [24] is the closest transform-based reverse-MIPS design. It combines a shifting-aware transform with hashing and a cone tree.

*Graph indexes for MIPS.* Forward-MIPS indexes optimize transformations, graph construction, or navigation [11, 12, 28, 33, 38, 41, 42]. They can serve as candidate backbones for query-centered top- $k$  MIPS, but they do not encode the rank-dependent acceptance boundary of each base vector for reverse reporting.

*Neighbor-distance modeling and compression.* Prior work models vector-specific  $k$ -distance curves to reduce storage or support a query-time rank. Achtert et al. fit a log-log power model [1]. This approximation is distinct from their exact metric-tree method [2]. Berrendorf et al. learn compact bounds with safe refinement [9]. Obermeier et al. use nonlinear approximation [34], and HAMG [36] extrapolates local distances during pruning. These results motivate compact modeling of rank-dependent neighbor thresholds. Low-rank approximation and vector quantization provide complementary compression tools [13, 16, 22, 23, 25]. ANQI applies these principles to the vector-by-rank matrix of scalar thresholds. Its verifier combines a low-rank model with learned packed residual quantization. This representation avoids a dense full-precision

threshold table. The model coefficients and quantized codes participate only in verification and never in graph routing. The result is a compact verifier integrated with a reverse-aware shared candidate geometry.

### 3 Problem formulation

Let  $\mathcal{X} = \{o_1, \dots, o_n\} \subset \mathbb{R}^D$  be a collection of  $n$  base vectors. We use  $\langle \cdot, \cdot \rangle$  for inner product,  $\|\cdot\|$  for Euclidean norm, and  $\delta(x, y) = \|x - y\|$  for Euclidean distance.

**Definition 1 (Reverse  $k$ -NN).** For a query  $q \in \mathbb{R}^D$  and requested rank  $k$ , let  $r_k(o)$  be the distance from  $o \in \mathcal{X}$  to its  $k$ -th nearest vector in  $\mathcal{X} \setminus \{o\}$ , and let  $\text{ip}_k(o)$  be its  $k$ -th largest inner product with a vector in  $\mathcal{X} \setminus \{o\}$ . Threshold ties are included. The reverse- $k$ NN answer  $A_k(q)$  contains every base vector for which  $q$  is among the  $k$  best matches. Under  $L_2$  distance,

$$A_k(q) = \text{RkNN}(q) = \{o \in \mathcal{X} : \delta(q, o)^2 \leq r_k(o)^2\}. \quad (2)$$

Under inner product,

$$A_k(q) = \text{RkNN}(q) = \{o \in \mathcal{X} : \langle q, o \rangle \geq \text{ip}_k(o)\}. \quad (3)$$

The answer size is data-dependent and need not equal  $k$ .

*One predicate for both metrics.* The two definitions differ only in the direction of their native scores. We orient both as a smaller-is-better threshold test so that both metrics share one predicate and query interface:

$$(\kappa(q, o), t_k(o)) = \begin{cases} (\delta(q, o)^2, r_k(o)^2), & L_2, \\ (-\langle q, o \rangle, -\text{ip}_k(o)), & \text{inner product.} \end{cases} \quad (4)$$

In either metric,

$$o \in A_k(q) \iff \kappa(q, o) \leq t_k(o). \quad (5)$$

This notation does not change the query. It only gives the two reverse predicates a common form.

*Approximate indexed query.* Evaluating (5) for every base vector requires a full scan. An approximate index avoids this cost with two stages. Given  $(q, k)$ , candidate generation retrieves a small set  $C(q) \subseteq \mathcal{X}$  that should contain the true reverse neighbors. Verification then tests only these candidates. It uses a represented cutoff  $\hat{t}_k(o)$  and returns

$$\hat{R}_k(q) = \{o \in C(q) : \kappa(q, o) \leq \hat{t}_k(o)\}. \quad (6)$$

This decomposition creates two independent error paths. A true reverse neighbor that is missing from  $C(q)$  cannot be recovered by verification. This error lowers recall. An inaccurate  $\hat{t}_k(o)$  can reject a true answer or accept a false one. This error affects recall or precision. The returned set is exact only when  $C(q)$  covers  $A_k(q)$  and  $\hat{t}_k(o) = t_k(o)$  for every candidate. We therefore measure the two errors separately.

*Evaluation protocol.* For a query collection  $\mathcal{Q}$ , we report retrieval quality against online and offline cost. Online cost consists of QPS and graph-distance evaluations; offline cost consists of construction time and persistent storage. Let  $E_{\text{graph}}(q)$  count the graph-space distance evaluations made while traversing the index. We report their average per query as

$$\bar{E}_{\text{graph}} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} E_{\text{graph}}(q). \quad (7)$$

**Table 2: Principal notation used throughout the paper. The unified predicate is  $\kappa(q, o) \leq t_k(o)$ ; hats denote quantities reconstructed by the compact verifier.**

| Symbol                                     | Meaning                                                           |
|--------------------------------------------|-------------------------------------------------------------------|
| $\mathcal{X}, n, D$                        | Base-vector collection, cardinality, and dimension.               |
| $q, o, k, \mathcal{K}$                     | Query, base vector, target rank, and represented rank set.        |
| $\kappa(q, o), t_k(o)$                     | Oriented query score and exact rank-specific cutoff.              |
| $\hat{t}_k(o)$                             | Reconstructed rank-specific cutoff.                               |
| $A_k(q), C(q), \hat{R}_k(q)$               | Exact answer, candidate set, and returned answer.                 |
| $T, \Theta, M, b$                          | Threshold matrix, verifier state, basis rank, and residual width. |
| $K_{\text{pivot}}, G, L$                   | Construction rank, graph degree, and descent width.               |
| $\mathcal{G}, \mathcal{E}, \mathcal{V}, I$ | Graph, entry structure, verifier state, and index.                |
| $Q, E_{\text{graph}}(q)$                   | Query batch and graph-distance evaluations per query.             |

This is the average number of graph-distance evaluations per query. Verifier reads and threshold preprocessing are excluded because they are not graph distance evaluations; QPS is measured separately under section 6. On the canonical million-scale datasets, exhaustive base-to-base computation supplies exact thresholds and exact RkNN answers. Earlier approximate  $L_2$  indexes primarily compare recall with query cost [10, 35, 36, 40], but recall alone can reward an approximate verifier for returning extra vectors. We therefore report both recall and precision.

Let  $Q_+ = \{q \in Q : |A_k(q)| > 0\}$ . For  $q \in Q_+$ , write  $A = A_k(q)$  and  $\hat{R} = \hat{R}_k(q)$ . The outcome counts are

$$\text{TP}_q = |A \cap \hat{R}|, \quad \text{FP}_q = |\hat{R} \setminus A|, \quad \text{FN}_q = |A \setminus \hat{R}|. \quad (8)$$

The per-query quality measures are

$$R_q = \frac{\text{TP}_q}{\text{TP}_q + \text{FN}_q}, \quad P_q = \frac{\text{TP}_q}{\text{TP}_q + \text{FP}_q}. \quad (9)$$

We define MacroR and MacroP as the means of  $R_q$  and  $P_q$ , respectively, over  $q \in Q_+$ . If  $\hat{R}_k(q)$  is empty, we set  $P_q = 1$  because it contains no false positive; recall and F1 are zero. F1 is computed per query and then averaged. Macro averaging gives each query equal weight, regardless of reverse-answer-set size.

## 4 ANQI

ANQI is a two-stage reverse-neighbor index. Proximity lifting moves the object-specific reverse cutoff into the indexed representation, so a forward ANN graph can generate high-coverage candidates. A compact verifier then reconstructs the requested target- $k$  cutoff and filters those candidates. Figure 3 summarizes index construction and query processing.

### 4.1 Lifting for Candidate Generation

Forward- $k$ NN graph indexes such as HNSW [30], NSG [18], and DiskANN [37] have made query-centered navigation a mature and efficient retrieval substrate. ANQI takes this demonstrated navigation capability as its starting point. However, applying it directly to reverse search fails: native distance or similarity gives a single ordering around  $q$ , whereas reverse membership is object-centered and also depends on a different acceptance cutoff for every base vector. ANQI bridges this mismatch by making the cutoff part of the indexed vector, so objects likely to accept  $q$  are visited early by a forward graph traversal. We first expose this structure for a fixed

$k$ . Under  $L_2$ , expanding the squared distance gives

$$\begin{aligned} \|q - o\|^2 \leq r_k(o)^2 &\iff \|q\|^2 + \|o\|^2 - 2 \cdot \langle q, o \rangle \leq r_k(o)^2 \\ &\iff \langle q, o \rangle - \frac{1}{2} \cdot (\|o\|^2 - r_k(o)^2) \geq \frac{1}{2} \cdot \|q\|^2. \end{aligned} \quad (10)$$

The rearrangement has the desired form. The left side combines the query-object interaction with the acceptance radius of  $o$ . The right side is identical for all objects considered by the query. Thus, the varying acceptance radii can influence candidate ranking instead of being consulted only after candidate discovery. Inner product admits the same separation:

$$\langle q, o \rangle \geq \text{ip}_k(o) \iff \langle q, o \rangle - \text{ip}_k(o) \geq 0. \quad (11)$$

In both metrics, the reverse predicate is now a comparison between one stored object-side score and one query-only cutoff. The score consists of the native query-object interaction plus a scalar correction known at construction time. ANQI stores that correction as one additional object coordinate and pairs it with a constant query coordinate:

$$o_k^\ell = \begin{cases} [o, -\frac{1}{2} \cdot (\|o\|^2 - r_k(o)^2)], & L_2, \\ [o, -\text{ip}_k(o)], & \text{inner product}, \end{cases} \quad q^\ell = [q, 1]. \quad (12)$$

The inner product of these lifted vectors reproduces the reverse-aware score without an object-specific operation at query time. Let  $s_k(q, o) = \langle q^\ell, o_k^\ell \rangle$ . Equations (10) and (11) then give

$$\begin{aligned} o \in A_k(q) &\iff s_k(q, o) \geq \tau(q), \\ \tau(q) &= \begin{cases} \frac{1}{2} \cdot \|q\|^2, & L_2, \\ 0, & \text{inner product}. \end{cases} \end{aligned} \quad (13)$$

The vector-specific threshold is now encoded in  $o_k^\ell$ . The remaining cutoff depends only on  $q$ . Reverse membership has become a forward maximum-inner-product test.

We use the standard XBOX reduction [7] to expose the reverse-aware score to Euclidean ANN search. The reverse-specific step is the preceding threshold lift, which places each object’s rank-dependent acceptance cutoff in the routing geometry. Choose  $M_\star \geq \max_{o \in \mathcal{X}} \|o_k^\ell\|$  and define

$$\bar{o}_k = \left[ o_k^\ell, \sqrt{M_\star^2 - \|o_k^\ell\|^2} \right], \quad \bar{q} = [q^\ell, 0]. \quad (14)$$

Every  $\bar{o}_k$  has norm  $M_\star$ , and

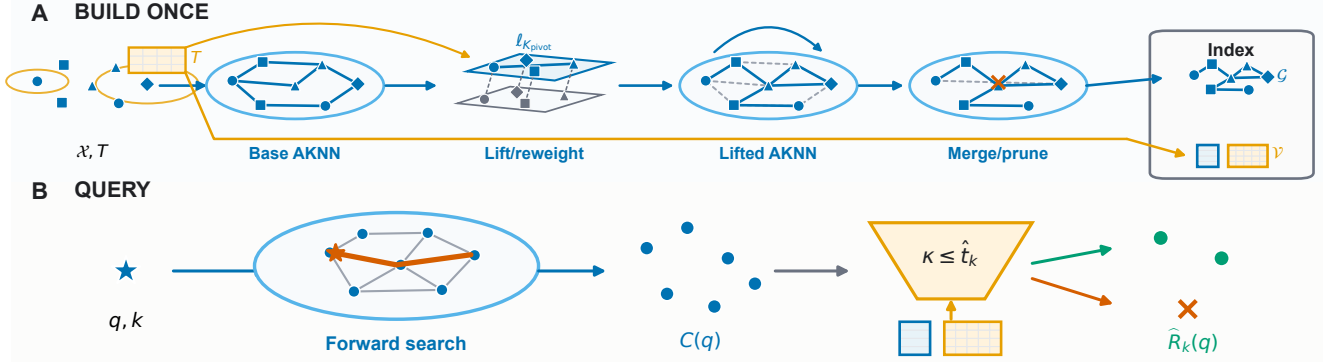
$$\|\bar{q} - \bar{o}_k\|^2 = \|q^\ell\|^2 + M_\star^2 - 2 \cdot s_k(q, o). \quad (15)$$

A larger reverse-aware score is therefore exactly equivalent to a shorter Euclidean distance. The membership test becomes

$$o \in A_k(q) \iff \|\bar{q} - \bar{o}_k\|^2 \leq \underbrace{\|q^\ell\|^2 + M_\star^2 - 2 \cdot \tau(q)}_{\rho_k(q)^2}. \quad (16)$$

**THEOREM 1 (EXACT REVERSE-AWARE CANDIDATE GEOMETRY).** *Assume exact target- $k$  thresholds and real arithmetic. For either  $L_2$  or inner product, the construction above has three properties. First, the lifted ball test in (16) is equivalent to exact reverse membership. Second, ordering base vectors by increasing  $\|\bar{q} - \bar{o}_k\|$  is equivalent to ordering them by decreasing reverse-aware score  $s_k(q, o)$ . Third, filtering any candidate set  $C(q)$  in the lifted space returns exactly  $C(q) \cap A_k(q)$ .*

*Proof sketch.* The score equivalences and norm equalization give the first two claims pointwise; intersecting the pointwise predicate



**Figure 3: ANQI workflow.** Construction reuses a base-space AKNN warm start, lifts and reweights its IDs, refines the lifted graph, and prunes it; thresholds independently encode the compact verifier. At query time, graph search returns  $C(q)$  and the verifier produces  $\hat{R}_k(q)$ . Repeated marker shapes track reused IDs and dashed edges inherited proposals. Blue shows graph operations; orange shows threshold data and verification.

with  $C(q)$  gives the third. The complete algebraic proof appears in Appendix A.1.

The lift replaces many vector-centered acceptance balls with one query-centered ordering that a forward ANN graph can navigate. It preserves the target- $k$  predicate for candidates reached by the graph, while the verifier makes the final membership decision.

## 4.2 One Graph Across Target Ranks

ANQI separates the graph-construction rank from the query rank. Choose a construction rank  $K_{\text{pivot}}$  and build the lifted graph from  $t_{K_{\text{pivot}}}(o)$ . This choice is useful because reverse answer sets grow monotonically with the target rank.

**PROPOSITION 1 (SHARED-RANK CANDIDATE ENVELOPE).** *For exact thresholds in either base space and any  $K \geq k$ ,*

$$A_k(q) \subseteq A_K(q). \quad (17)$$

*Consequently, the exact lifted acceptance region at  $K$  is a candidate superset for every target rank no larger than  $K$ .*

*Proof sketch.* Neighbor radii are nondecreasing in  $k$ , while the  $k$ th inner-product order statistic is nonincreasing; the lifted geometry preserves these pointwise predicates. See Appendix A.1.

For each represented  $k \leq K_{\text{pivot}}$ , the construction-rank lift supplies a shared candidate envelope, and the target- $k$  verifier removes its extra members. In the evaluated  $L_2$  cross-pivot setting, query-time collection substitutes the decoded target- $k$  cutoff while retaining the graph edges; fig. 9 reports the resulting measured robustness.

**LEMMA 1 (TARGET-RANK SCORE CORRECTION).** *For the  $L_2$  lift and any represented threshold  $\tilde{t}_k(v)$ , define*

$$s_{\tilde{t}_k}(q, v) = \langle q, v \rangle - \frac{1}{2} \cdot (\|v\|^2 - \tilde{t}_k(v)). \quad (18)$$

*Then, for every target rank  $k$ ,*

$$s_{K_{\text{pivot}}}(q, v) + \frac{1}{2} \cdot (\tilde{t}_k(v) - t_{K_{\text{pivot}}}(v)) = s_{\tilde{t}_k}(q, v). \quad (19)$$

*In particular, setting  $\tilde{t}_k = t_k$  gives the exact target- $k$  predicate, independently of whether  $k \leq K_{\text{pivot}}$ .*

*Proof sketch.* Substitution cancels the construction-rank threshold term and leaves  $s_{\tilde{t}_k}(q, v)$ ; see Appendix A.1.

The lemma isolates the beyond-pivot issue: score correction and verification are exact for an exact target threshold, whereas finite-beam graph traversal must still reach the newly admitted part of the target envelope. For any candidate set  $C_k(q)$  and returned set  $\hat{R}_k(q) \subseteq C_k(q)$ , this separation can be written exactly as

$$\text{Recall}_q = \underbrace{\frac{|A_k(q) \cap C_k(q)|}{|A_k(q)|}}_{\text{candidate coverage}} \cdot \underbrace{\frac{|A_k(q) \cap \hat{R}_k(q)|}{|A_k(q) \cap C_k(q)|}}_{\text{verifier retention}}, \quad (20)$$

whenever the first denominator in the second factor is nonzero (and the product is defined as zero otherwise). Thus the missing beyond-pivot theorem concerns the first factor, not the algebraic correctness of the target-rank verifier.

## 4.3 Local Radius Law for Euclidean Space

Verification needs one vector-specific threshold at the requested rank. Storing a full-precision scalar for every vector and rank is memory intensive. For Euclidean data, this threshold is  $t_k(o) = r_k(o)^2$ . ANQI instead compresses the vector-by-rank threshold matrix. Prior RkNN indexes have shown that  $k$ -distance curves admit compact functional or learned approximations [1, 9, 34], while HAMG [36] estimates local neighbor distances during query processing. ANQI builds on this established observation but uses a different representation for independent post-filter verification. Let  $T[o, k] = t_k(o)$  be the exact vector-by-rank threshold matrix. ANQI represents this matrix as

$$t_k(o) = \mu_k + \sum_{m=1}^M c_m(o) \cdot g_{m,k} + e_{o,k}. \quad (21)$$

The shared basis  $g_{m,k}$  captures regular variation across ranks. Each vector stores  $M$  coefficients  $c_m(o)$  and a packed residual code. We call this learned, rank-specific encoding of  $e_{o,k}$  learned residual quantization (LRQ). Before a query batch, ANQI decodes the requested threshold column. For each rank, the ordered codebook  $\mathcal{B}_k$  includes a lower endpoint no larger than every encoded residual,

and the raw floor decoder is

$$\begin{aligned} \tilde{e}_{o,k} &= \max\{b \in \mathcal{B}_k : b \leq e_{o,k}\}, \\ \hat{t}_k(o) &= \mu_k + \sum_{m=1}^M c_m(o) \cdot g_{m,k} + \tilde{e}_{o,k}. \end{aligned} \quad (22)$$

Verification then needs one scalar read per candidate. The deployed configuration uses  $M = 8$  and learned u8 residual quantization. A standalone control sets  $e_{o,k} = 0$  and fits an empirical M-only basis to isolate the shared low-rank term in section 6.5. For IP, the same representation is fitted directly to signed threshold quantiles; section 4.4 gives their interpretation.

The factorization is empirical, but  $L_2$  neighbor radii have a classical local model. Nearest-neighbor density estimation relates the mass of the  $k$ th-neighbor ball to its volume [29]. Intrinsic-dimension estimation applies the same idea on a locally smooth manifold: close-neighbor arrivals are approximated by a Poisson process whose rate scales with the surface area of a  $d$ -dimensional ball [27]. Absorbing the finite-sample intensity into  $\rho(o)$  gives the local ball-mass model  $N_o(r) = \rho(o) \cdot V_{d(o)} \cdot r^{d(o)}$ , where  $d(o)$  may vary with the object. This approximation is local in scale. Curvature, changing density, or a change in intrinsic dimension can prevent one power law from fitting a wide rank interval.

**PROPOSITION 2 (EUCLIDEAN LOCAL RADIUS LAW).** *Under the local ball-mass model,*

$$r_k(o)^2 = \gamma(o) \cdot k^{v(o)}, \quad v(o) = \frac{2}{d(o)}, \quad \gamma(o) = [\rho(o) \cdot V_{d(o)}]^{-v(o)}. \quad (23)$$

For any reference exponent  $v_0$ , write  $v(o) = v_0 + \beta(o)$ . Then

$$r_k(o)^2 = \sum_{m=0}^{\infty} \underbrace{\frac{\gamma(o) \cdot \beta(o)^m}{m!}}_{f_m(o)} \underbrace{k^{v_0} \cdot (\ln k)^m}_{\phi_m(k)}. \quad (24)$$

For the degree- $p$  truncation, the remainder satisfies

$$|R_p(o, k)| \leq \gamma(o) \cdot k^{v_0} \cdot e^{|\beta(o) \cdot \ln k|} \cdot \frac{|\beta(o) \cdot \ln k|^{p+1}}{(p+1)!}. \quad (25)$$

Thus a rank interval on which  $|\beta(o) \cdot \ln k|$  remains moderate admits a small shared basis; the bound does not require the ambient dimension  $D$  to be small.

*Proof sketch.* Solve the local mass equation for  $r_k(o)^2$ , expand  $e^{\beta(o) \cdot \ln k}$ , and apply the standard exponential Taylor remainder bound. The full derivation is in Appendix A.2.

The law separates the two local effects. At fixed  $d(o)$ , increasing local intensity shrinks the radius as  $\rho(o)^{-1/d(o)}$ , while increasing the requested rank grows it as  $k^{1/d(o)}$ . Equivalently,

$$\log r_k(o) = \frac{1}{d(o)} [\log k - \log(\rho(o) \cdot V_{d(o)})].$$

The log-radius slope is therefore the inverse local dimension on a scale where the model holds. Different objects may have different slopes and intercepts, which is why ANQI needs an object-specific curve rather than one global radius schedule.

**COROLLARY 1 (LOCAL SHELL GROWTH BEYOND THE CONSTRUCTION RANK).** *Under the same local ball-mass model, for  $k \geq K$ ,*

$$\frac{r_k(o)^2}{r_K(o)^2} = \left(\frac{k}{K}\right)^{v(o)}, \quad N_o(r_k(o)) - N_o(r_K(o)) = k - K. \quad (26)$$

Consequently, increasing the target rank expands the local acceptance shell by a mass proportional to  $k - K$ , while its squared-radius inflation is governed by the local intrinsic dimension through  $v(o) = 2/d(o)$ .

*Proof sketch.* Divide the Local Radius Law at ranks  $k$  and  $K$  and use  $N_o(r_j(o)) = j$  for  $j \in \{k, K\}$ ; see Appendix A.2.

The Local Radius Law provides a Euclidean rationale for why  $L_2$  thresholds at nearby ranks can share a small basis. ANQI learns that basis directly from the exact threshold matrix, allowing the learned components and residuals to capture departures from the local model.

#### 4.4 Empirical Threshold Modeling for Inner Product

The Local Radius Law is specific to Euclidean radii. IP thresholds instead form directional score quantiles, and ANQI fits them with the same low-rank-plus-LRQ verifier layout.

**PROPOSITION 3 (PROJECTION QUANTILES FOR INNER PRODUCT).** *For a nonzero object  $o$ , let  $u_o = o/\|o\|$ , and let  $\pi_k(o)$  be the  $k$ th largest value in  $\{\langle x, u_o \rangle : x \in \mathcal{X} \setminus \{o\}\}$ . Then*

$$\text{ip}_k(o) = \|o\| \cdot \pi_k(o), \quad o \in A_k(q) \iff \langle q, u_o \rangle \geq \pi_k(o). \quad (27)$$

Equivalently, the signed verifier row is  $T[o, k] = -\|o\| \cdot \pi_k(o)$ . If  $o = 0$ , the row is identically zero.

*Proof sketch.* Normalize  $o$  and factor the positive scalar  $\|o\|$  out of each inner product; the zero-vector case is immediate. See Appendix A.3.

Across ranks, each signed IP row is the negative upper-quantile curve of projections onto  $u_o$ . ANQI learns a shared basis directly from the exact signed threshold matrix, and LRQ stores the remaining object- and rank-specific deviation.

#### 4.5 Verification Guarantees and Controls

The reconstructed cutoff determines which type of verification error is possible. Let  $\hat{A}_k(q) = \{o : \kappa(q, o) \leq \hat{t}_k(o)\}$  denote the answer obtained by applying the represented threshold to the full base collection.

**PROPOSITION 4 (ONE-SIDED THRESHOLD GUARANTEES).** *Assume exact target thresholds. If  $\hat{t}_k(o) \leq t_k(o)$  for every  $o$ , then  $\hat{A}_k(q) \subseteq A_k(q)$  and verification introduces no false positives. Its false negatives are confined to*

$$\{o : \hat{t}_k(o) < \kappa(q, o) \leq t_k(o)\}. \quad (28)$$

*If  $\hat{t}_k(o) \geq t_k(o)$  for every  $o$ , then  $A_k(q) \subseteq \hat{A}_k(q)$  and verification introduces no false negatives.*

*Proof sketch.* Chain the represented and exact threshold inequalities with  $\kappa(q, o)$ ; the complete boundary argument is given in Appendix A.4.

By (22), the raw floor-decoded threshold satisfies  $\hat{t}_k(o) \leq t_k(o)$  in real arithmetic. Before any optional query-time threshold adjustment, LRQ is therefore precision-oriented: its compression error cannot introduce false positives, while its boundary band can lower recall.

The local ball-mass model also relates threshold error to answer quality. Let  $d_{\max} = \max_o d(o)$ . If an exact-source  $L_2$  verifier is

restrictive and

$$\epsilon_k = \max_o \frac{r_k(o)^2 - \hat{r}_k(o)^2}{r_k(o)^2} < 1,$$

then its full-collection pooled recall satisfies

$$\text{Recall}_{\text{pool}} \geq (1 - \epsilon_k)^{d_{\max}/2}. \quad (29)$$

The bound follows because local ball mass scales as  $r^{d(o)}$ . It quantifies the verifier component; the graph-geometry experiments separately measure candidate coverage. Appendix A.6 makes this connection explicit by relating the bound to LRQ codebook spacing and by factoring end-to-end recall into candidate coverage and verifier retention.

*M-only reconstruction.* The empirical basis contains one shared column  $g_{\cdot k}$  for every configured rank. With residuals disabled, each object therefore stores only its  $M$  coefficients and reconstructs  $\hat{t}_k(o) = \mu_k + \sum_m c_m(o) \cdot g_{m,k}$ . This control exposes how much of the object-specific threshold curve is captured by the shared low-rank model before rank-specific correction. Its error is unconstrained and may be positive or negative. The floor-quantized residual restores the one-sided raw LRQ reconstruction. Section 6.5 measures its reconstruction error and predicate quality over the configured Euclidean rank range.

The constructions above produce the two artifacts used by ANQI. Equations (12) and (14) define graph coordinates  $\bar{o}$  and the corresponding query map  $\Lambda$ . Equations (21) and (22) define the compact verifier state. Section 5 shows how construction builds the graph from the first artifact, encodes the second, and combines them only when verifying retrieved candidates.

## 5 Index Realization

ANQI follows one dataflow. A base-space AKNN computation first supplies plausible neighbor IDs. Algorithm 1 reweights and refines that warm start in the lifted space, then encodes the threshold verifier. At query time, algorithm 2 traverses the graph, collects a variable-size candidate set, and applies the requested target- $k$  predicate. The graph-search width controls descent effort; it does not fix the number of returned vectors.

### 5.1 From base-space AKNN to the lifted graph

Construction is a warm-started two-space pipeline: base-space AKNN, lifting, lifted-distance reweighting, lifted-space AKNN refinement, and bounded-degree pruning. The second AKNN stage retains the base-space neighbor IDs, recomputes their edge weights in the lifted geometry, and refines them with NN-Descent [15]. The final adjacency is therefore selected in the lifted metric rather than copied from the base-space graph.

Each  $\mathcal{W}$  or  $\mathcal{H}$  is an  $n$ -row neighbor table. The released builder seeds  $\mathcal{W}_{\text{base}}$  from random-projection-tree leaves, adds reverse edges, and uses beam gathering to retain nearby base-space IDs. After reweighting, each lifted-space local-join pass pairs sampled neighbors of the same vertex and inserts a pair when it improves either neighbor table. Refinement stops after six passes or when a pass changes fewer than  $n/200$  entries. Reverse-edge gathering then forms a candidate pool per vertex, and RNG-style pruning keeps at most  $G$  outgoing edges. A single random-projection tree maps each query to a leaf medoid used only as the graph entry point.

### Algorithm 1 ANQI index construction.

---

**Require:** metric  $m$ ; base vectors  $\mathcal{X}$ ; represented ranks  $\mathcal{K}$ ; exact thresholds  $T[o, k]$ ; construction rank  $K_{\text{pivot}} \in \mathcal{K}$ ; maximum graph degree  $G$ ; trained verifier  $\Theta = (\mu, \{g_m\}_{m=1}^M, \{\mathcal{B}_k\}_{k \in \mathcal{K}})$

- 1: **procedure** BUILDINDEX( $\mathcal{X}, T, \Theta, \mathcal{K}, K_{\text{pivot}}, G, m$ )
- 2:    $\mathcal{W}_{\text{base}} \leftarrow \text{APPROXKNN}(\mathcal{X}, m)$  ▷ base-space AKNN warm start
- 3:   **for all**  $o \in \mathcal{X}$  **in parallel do**
- 4:      $z_o \leftarrow \text{REVERSELIFT}(o, T[o, K_{\text{pivot}}], m)$  ▷ (12)
- 5:   **end for**
- 6:    $(\{\bar{o}\}, \Lambda) \leftarrow \text{EQUALIZENORMS}(\{z_o\})$  ▷ (14)
- 7:    $\mathcal{W}_L \leftarrow \text{REWEIGHTWARMSTART}(\mathcal{W}_{\text{base}}, \{\bar{o}\})$  ▷ same IDs; lifted distances
- 8:    $\mathcal{H} \leftarrow \mathcal{W}_L$
- 9:   **for**  $j = 1, \dots, 6$  **do**
- 10:      $(\mathcal{H}, \Delta_j) \leftarrow \text{LOCALJOIN}(\mathcal{H}, \{\bar{o}\})$  ▷ NN-Descent updates
- 11:     **if**  $\Delta_j < n/200$  **then**
- 12:       **break**
- 13:     **end if**
- 14:   **end for**
- 15:    $\mathcal{P} \leftarrow \text{ADDREVERSEANDBEAMGATHER}(\mathcal{H}, \{\bar{o}\})$
- 16:    $\mathcal{G} \leftarrow \text{REVERSESCATTERANDRNGPRUNE}(\mathcal{P}, G)$
- 17:    $\mathcal{E} \leftarrow \text{BUILDRPTREE}(\{\bar{o}\})$
- 18:   **for all**  $o \in \mathcal{X}$  **in parallel do**
- 19:     **for**  $m = 1, \dots, M$  **do**
- 20:        $c_m(o) \leftarrow \langle T[o, :] - \mu, g_m \rangle$
- 21:     **end for**
- 22:     **for all**  $k \in \mathcal{K}$  **do**
- 23:        $a_{o,k} \leftarrow \mu_k + \sum_{m=1}^M c_m(o) \cdot g_{m,k}$
- 24:        $u_{o,k} \leftarrow \text{FLOORENCODE}(T[o, k] - a_{o,k}, \mathcal{B}_k)$
- 25:     **end for**
- 26:   **end for**
- 27:    $\mathcal{V} \leftarrow (\mu, \{g_m\}, \{\mathcal{B}_k\}, \{c(o)\}, \{u_{o,k}\})$
- 28:   **return**  $\mathcal{I} = (\{\bar{o}\}, \mathcal{G}, \mathcal{E}, \Lambda, \mathcal{V}, K_{\text{pivot}}, m)$
- 29: **end procedure**

---

The graph uses only the construction-rank coordinate. The coefficient and residual arrays are stored in  $\mathcal{V}$  and cannot alter graph distances or edges. One graph can therefore generate candidates for several target ranks while the verifier changes the final decision.

The exact threshold table and trained verifier model are construction inputs. On a fixed sample of threshold rows, training starts from a rank- $M$  PCA basis and uses three rounds that alternate per-rank one-dimensional Lloyd codebooks with basis refitting. Construction projects every full threshold row onto the resulting orthonormal basis, extends each ordered codebook to the full residual range, and floor-encodes the residual through (22). The persistent index keeps the coefficients and codes, not the full table  $T$ .

### 5.2 Query processing

Algorithm 2 exposes the two query stages. The decoder first materializes one target-rank threshold column for the whole batch. Each query then uses bounded graph descent to reach the relevant lifted neighborhood. A second frontier follows the connected part of the candidate envelope, and the verifier tests the resulting candidates in the base space.

The two queues have different jobs. The bounded queue supplies robust navigation even when the entry point lies outside the acceptance region. Once a visited vertex passes the candidate-envelope test, the FIFO follows its accepted component without a top- $L$  cardinality cap. For  $k \leq K_{\text{pivot}}$ , this test is the construction-rank envelope certified by proposition 1; the target- $k$  verifier removes its extra members. For the evaluated  $L_2$  cross-pivot path, collection uses the decoded target-rank threshold on the fixed graph topology.

Concretely, for  $k \leq K_{\text{pivot}}$ , INCANDIDATEENVELOPE applies the construction-rank lifted-ball test in (16). For the evaluated  $L_2$  path

---

**Algorithm 2** ANQI batched reverse- $k$ NN search.

---

**Require:** index  $I$ ; original vectors  $X$ ; query batch  $Q$ ; requested rank  $k \in \mathcal{K}$ ; descent width  $L$

```

1: procedure SEARCHBATCH( $I, X, Q, k, L$ )
2:    $(\hat{t}_k(o))_{o \in \mathcal{X}} \leftarrow \text{DECODECOLUMN}(\mathcal{V}, k)$  ▷ once per batch
3:   for all  $q \in Q$  in parallel do
4:      $\bar{q} \leftarrow \Lambda(q); e \leftarrow \text{ENTRYPOINT}(\mathcal{E}, \bar{q})$ 
5:      $Q_b \leftarrow \text{BOUNDEDQUEUE}(L); Q_f \leftarrow \text{FIFO}; C(q) \leftarrow \emptyset; \text{seen} \leftarrow \emptyset$ 
6:     EXPLORE( $e, \text{true}$ )
7:     while  $Q_b$  has an unexpanded item do
8:        $u \leftarrow \text{CLOSESTUNEXPANDED}(Q_b)$ 
9:       for all  $v \in N_G(u)$  do
10:        EXPLORE( $v, \text{true}$ )
11:       end for
12:     end while
13:     while  $Q_f$  is not empty do
14:        $u \leftarrow \text{POP}(Q_f)$ 
15:       for all  $v \in N_G(u)$  do
16:        EXPLORE( $v, \text{false}$ )
17:       end for
18:     end while
19:      $\hat{R}_k(q) \leftarrow \{o \in C(q) : \kappa(q, o) \leq \hat{t}_k(o)\}$ 
20:   end for
21:   return  $\{\hat{R}_k(q) : q \in Q\}$ 
22: end procedure
23: function EXPLORE( $u, \text{descend}$ ) ▷ query-local state
24:   if  $v$  is unseen then
25:     mark  $v$ ;  $d_v \leftarrow \|\bar{q} - v\|^2$ 
26:     if descend then
27:       INSERTBOUNDED( $Q_b, (d_v, v), L$ )
28:     end if
29:     if INCANDIDATEENVELOPE( $v, d_v, q, k, I, \hat{t}_k(v)$ ) then
30:       PUSH( $Q_f, v$ );  $C(q) \leftarrow C(q) \cup \{v\}$ 
31:     end if
32:   end if
33: end function

```

---

with  $k > K_{\text{pivot}}$ , it uses the score adjustment

$$s_{K_{\text{pivot}}}(q, v) + \frac{1}{2} \cdot (\hat{t}_k(v) - t_{K_{\text{pivot}}}(v)) \geq \frac{1}{2} \cdot \|q\|^2, \quad (30)$$

The adjustment replaces the construction-rank cutoff in  $s_{K_{\text{pivot}}}(q, v)$  with  $\hat{t}_k(v)$ , thereby evaluating the represented target- $k$  envelope without changing graph edges; its algebraic identity is given by lemma 1. The evaluated IP configuration uses represented ranks  $k \leq K_{\text{pivot}}$ . The construction threshold in (30) is already encoded in the stored lift; the discarded full table  $T$  is not consulted.

The final line evaluates squared  $L_2$  distance or signed inner product against the materialized threshold. It does not consult the exact table. A reusable version stamp implements seen without clearing it for every query. The entry structure chooses a starting vertex only; it does not define graph edges.

### 5.3 Computation and storage complexity

Construction work has three parts: the base-space warm start, lifted-graph building, and verifier encoding. Let  $T_{\text{AKNN}}$  be the base-space AKNN time and  $T_{\mathcal{E}}$  the entry-tree construction time. The lifted-graph stage evaluates  $E_{\text{build}}$  vector pairs and performs  $P_{\text{build}}$  updates on temporary candidate queues of width  $B_{\text{build}}$ . Given  $T$  and a trained  $\Theta$ , the total cost is

$$\begin{aligned} & O(T_{\text{AKNN}} + T_{\mathcal{E}} + n \cdot D + E_{\text{build}} \cdot D \\ & + P_{\text{build}} \cdot \log B_{\text{build}} \\ & + n \cdot |\mathcal{K}| \cdot M + n \cdot |\mathcal{K}| \cdot b). \end{aligned} \quad (31)$$

The  $n \cdot |\mathcal{K}| \cdot b$  term assumes binary search in each ordered  $2^b$ -entry codebook. Threshold computation and model fitting are preprocessing costs and are not hidden inside this bound. Here  $T_{\text{AKNN}}$  covers only the base-space warm start; lifted-space reweighting, NN-Descent refinement, and pruning are charged to  $E_{\text{build}}$  and  $P_{\text{build}}$ .

Excluding the source base vectors shared by all methods, persistent index storage is

$$S_{\text{index}} = O\left(n \cdot (D + G + M) + |\mathcal{K}| \cdot M + |\mathcal{K}| \cdot 2^b\right) + n \cdot |\mathcal{K}| \cdot b \text{ bits}. \quad (32)$$

The first term is measured in machine-word storage units for lifted coordinates, bounded-degree adjacency, object coefficients, shared mean and basis values, and the codebooks. In particular, each  $2^b$ -entry codebook contributes  $2^b$  scalar codewords; it is not itself a machine word. The second term is the packed residual stream. The entry structure is linear and is absorbed by the first term. The exact  $n \cdot |\mathcal{K}|$  threshold table is transient and can be discarded after encoding. Decoding one rank costs  $O(n \cdot M)$  time and one transient  $n$ -scalar column, shared by the query batch.

For one query, let  $S_{\text{adj}}(q)$  be the number of adjacency entries scanned. The online cost is

$$O(E_{\text{graph}}(q) \cdot D + S_{\text{adj}}(q) + E_{\text{graph}}(q) \cdot \log L + |C(q)| \cdot D), \quad (33)$$

where the last term is base-space verification. The threshold-column cost amortizes to  $O(n \cdot M/|Q|)$  per query in a batch. A worker uses  $O(n + L + |C(q)|)$  reusable query state for the version stamps, bounded queue, and candidate output. In the worst case  $E_{\text{graph}}(q)$  and  $|C(q)|$  can reach  $n$ ; the index is effective when lifted routing keeps both far below a full scan.

## 6 Experimental Evaluation

We measure ANQI’s quality-throughput frontier, offline cost, and scaling behavior. Controlled experiments then separate the effects of threshold modeling and graph geometry.

### 6.1 Experimental setup

The benchmark covers both metrics, several embedding types, and three target ranks. The  $L_2$  datasets span 96-dimensional learned embeddings, 128-dimensional local descriptors, and 960-dimensional GIST vectors. The inner-product datasets contain recommendation and text embeddings. Each main dataset has approximately one million base vectors, and every method is tested with held-out queries at  $k \in \{10, 50, 100\}$ . SIFT1B appears only in the scalability study.

**Table 3: Datasets in the main benchmark. Distance is the reverse-query measure;  $n$  is the number of base vectors,  $D$  is the base-space dimension, and  $|Q|$  is the number of query vectors.**

| Dataset   | Source | Distance | $n$       | $D$ | $ Q $  |
|-----------|--------|----------|-----------|-----|--------|
| SIFT1M    | Image  | $L_2$    | 1,000,000 | 128 | 10,000 |
| DEEP1M    | Image  | $L_2$    | 1,000,000 | 96  | 10,000 |
| GIST1M    | Image  | $L_2$    | 1,000,000 | 960 | 1,000  |
| Music100  | Music  | IP       | 999,900   | 100 | 100    |
| MSMarco1M | Text   | IP       | 1,000,000 | 768 | 9,376  |

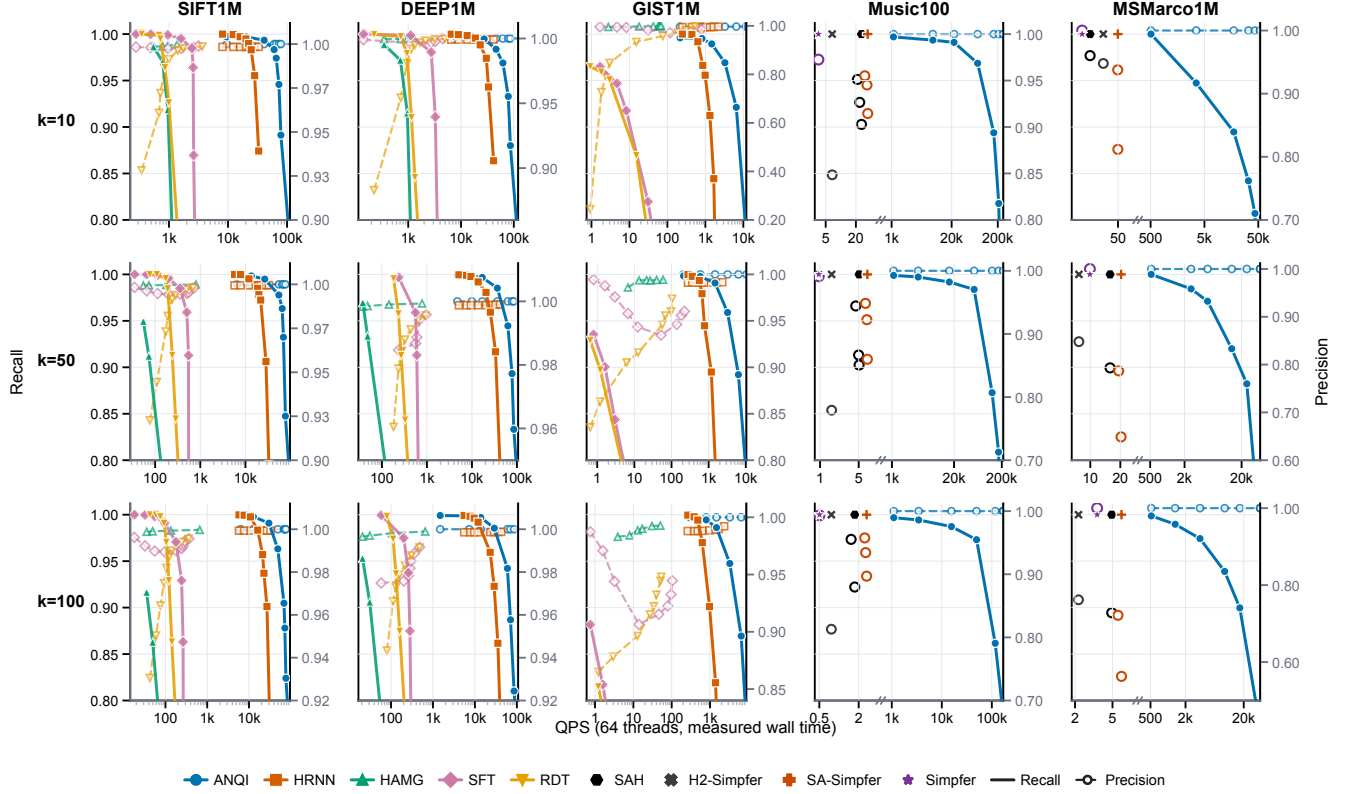


Figure 4: End-to-end recall, precision, and 64-thread QPS on five datasets. Rows are  $k = 10, 50, 100$ ; columns are three  $L_2$  and two IP datasets. Filled traces use the shared leftmost Recall axis, and hollow dashed traces use each panel’s right Precision axis. The IP panels use a marked axis break to omit the empty QPS interval between the low-throughput baselines and ANQI; both displayed segments remain logarithmic. Quality is measured against exact RkNN ground truth.

Table 4: Selected operating points from the complete frontiers in fig. 4, nearest recall 0.95 ( $L_2$ ) and 0.99 (IP). Cells show recall/precision/QPS/distance evaluations (R/P/Q/D) for  $L_2$  and R/P/Q for IP. Red brackets give ANQI’s QPS speedup over the underlined fastest recall-qualified baseline; ‡ marks a missed target.

| $L_2$ Dataset | $k$ | ANQI                            | HRNN [40]               | HAMG [36]               | SFT [35]                | RDT [10]                |
|---------------|-----|---------------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
| SIFT1M        | 10  | 0.954/1.000/71.8k/2.6k [2.57×]  | 0.951/0.998/27.9k/5.74k | 0.970/0.999/781/204k    | 0.964/0.998/2.57k/161k  | 0.964/0.987/855/349k    |
|               | 50  | 0.963/1.000/65.2k/2.88k [2.84×] | 0.958/0.999/23k/6.75k   | 0.949‡/0.999/54.8/4.48M | 0.959/0.995/504/1.29M   | 0.959/0.988/213/1.86M   |
|               | 100 | 0.963/1.000/49k/3.3k [2.84×]    | 0.957/0.999/17.2k/7.36k | 0.917‡/0.999/35.4/6.96M | 0.971/0.990/179/4.11M   | 0.970/0.982/105/4.81M   |
| DEEP1M        | 10  | 0.955/1.000/69k/2.84k [2.14×]   | 0.958/0.999/32.3k/6.85k | 0.972/1.000/709/212k    | 0.981/0.997/2.73k/253k  | 0.979/0.981/946/470k    |
|               | 50  | 0.958/1.000/55.2k/3.4k [2.15×]  | 0.959/0.999/25.6k/8.15k | 0.970/0.998/37.5/6.18M  | 0.959/0.987/557/1.5M    | 0.958/0.979/233/2.07M   |
|               | 100 | 0.954/1.000/54k/3.74k [2.34×]   | 0.960/0.999/23.1k/8.97k | 0.953/0.997/20.9/11.2M  | 0.975/0.976/201/4.99M   | 0.974/0.966/112/5.49M   |
| GIST1M        | 10  | 0.956/0.997/4.39k/9.07k [4.44×] | 0.956/0.995/987/22.5k   | 0.724‡/0.996/2.79/16.6M | 0.959/0.997/2.37/17.8M  | 0.951/0.847/3.01/31M    |
|               | 50  | 0.959/1.000/3.23k/12.4k [4.22×] | 0.967/0.991/766/31.7k   | 0.416‡/0.986/6.70/5.69M | 0.935‡/0.994/0.80/54.1M | 0.929‡/0.835/0.65/107M  |
|               | 100 | 0.963/1.000/2.66k/15.5k [3.86×] | 0.953/0.988/689/31.7k   | 0.361‡/0.983/3.92/10.2M | 0.882‡/0.987/0.75/54.4M | 0.815‡/0.865/1.21/47.2M |
| IP Dataset    | $k$ | ANQI                            | SAH [24]                | H2-S [3]                | SA-S [3]                | SIMP. [3]               |
| Music100      | 10  | 0.991/1.000/22.5k [654.2×]      | 1.000/0.903/25.8        | 1.000/0.849/6.74        | 1.000/0.915/34.5        | 1.000/0.973/3.58        |
|               | 50  | 0.992/1.000/15.9k [2.215×]      | 1.000/0.851/5.08        | 1.000/0.779/1.62        | 1.000/0.860/7.18        | 1.000/0.991/0.95        |
|               | 100 | 0.992/1.000/7.66k [2.895×]      | 1.000/0.880/1.74        | 1.000/0.813/0.76        | 1.000/0.897/2.65        | 1.000/0.993/0.50        |
| MSMarco       | 10  | 1.000/1.000/508 [10.19×]        | 1.000/0.960/32.1        | 1.000/0.948/39.6        | 1.000/0.812/49.9        | 1.000/1.000/28.2        |
|               | 50  | 0.998/1.000/973 [47.81×]        | 1.000/0.793/15.8        | 1.000/0.848/7.68        | 1.000/0.649/20.4        | 1.000/1.000/9.90        |
|               | 100 | 0.995/1.000/973 [156.2×]        | 1.000/0.728/4.91        | 1.000/0.762/2.18        | 1.000/0.563/6.23        | 1.000/1.000/3.43        |

**Baselines.** We group methods by their native reverse predicate. For  $L_2$  RkNN, SFT [35] filters a forward neighborhood, and Rdt [10] applies dimensional tests. HAMG [36] explores multiple graph hops, and HRNN [40] combines graph navigation with ranked  $k$ NN and reverse-neighbor lists. All four compare Euclidean query-to-object

distances with object-specific neighbor radii. The public HRNN [40] codebase supplies HRNN [40] and the HNSW-based reference reproductions used here for SFT [35], RDT [10], and HAMG [36]. For

**Table 5: Construction time and index footprint. ANQI includes model fitting; IP baselines use native indexing times and in-memory footprints. Other footprints are persisted. Boldface marks column minima per dataset.**

| $L_2$   |                       |                      | IP        |                |                      |
|---------|-----------------------|----------------------|-----------|----------------|----------------------|
| Dataset | Method                | Build Index (s) (GB) | Dataset   | Method         | Build Index (s) (GB) |
| SIFT1M  | ANQI                  | 102.23 0.85          | Music100  | ANQI           | 882.89 0.71          |
|         | HRNN [40]             | 150.00 6.67          |           | SAH [24]       | 17.73 0.50           |
|         | SFT/RDT [10, 35] HNSW | <b>14.00 0.66</b>    |           | H2-Simpfer [3] | 1.57 0.84            |
|         | HAMG [36]             | 69.00 20.53          |           | SA-Simpfer [3] | <b>1.45 0.47</b>     |
| DEEP1M  | ANQI                  | 88.78 0.73           | MSMarco1M | SIMPFER [3]    | 8.31 0.64            |
|         | HRNN [40]             | 152.00 6.54          |           | ANQI           | 545.87 3.47          |
|         | SFT/RDT [10, 35] HNSW | <b>12.00 0.53</b>    |           | SAH [24]       | 125.97 0.62          |
|         | HAMG [36]             | 51.17 20.40          |           | H2-Simpfer [3] | <b>5.61 0.86</b>     |
| GIST1M  | ANQI                  | 308.17 4.25          |           | SA-Simpfer [3] | 5.94 <b>0.47</b>     |
|         | HRNN [40]             | 480.00 10.00         |           | SIMPFER [3]    | 6.88 0.64            |
|         | SFT/RDT [10, 35] HNSW | <b>77.00 3.99</b>    |           |                |                      |
|         | HAMG [36]             | 308.00 23.86         |           |                |                      |

reverse MIPS, SIMPFER [3] prunes users with score bounds. H2-SIMPFER [3] and SA-SIMPFER [3] add H2 or shifting-aware transformations. Full SAH [24]/SA-Cone [24] further combines shifting-aware hashing with a cone tree. We run all four from the authors’ SAH repository. SA-SIMPFER is a controlled variant, whereas SAH/SA-Cone is the complete index.

**Protocol.** Fixed parameters retain the defaults of each released implementation. We tune only the documented control that exposes each method’s quality-cost trade-off. We compare at recall 0.95 for  $L_2$  and 0.99 for IP, matching the high-recall operating ranges of the corresponding baseline families. Figure 4 reports the complete frontiers around both targets. Each reported run uses one server with two 32-core Intel Xeon Platinum 8457C sockets, SMT disabled. Every method receives the same 64 physical cores, query set, and ground truth. ANQI and the  $L_2$  implementations parallelize across queries; the released IP implementations parallelize across query and user-block tasks. Both decompositions use the same total core budget and report whole-batch wall-time QPS.

For the five main datasets, exhaustive distance evaluation produces exact base-to-base top-100 thresholds and exact memberships for  $k = 10, 50, 100$ . ANQI uses one degree-48 graph constructed at  $K_{\text{pivot}} = 100$  and one rank-8 packed-u8 verifier; the same index serves all three target ranks. We report macro recall and precision from eq. (9). For  $L_2$ , D counts every online full-vector distance evaluation in candidate generation and verification, which is broader than the graph-distance count in eq. (7).

## 6.2 End-to-end effectiveness

Figure 4 shows the complete frontiers; table 4 lists the measured point nearest each recall target from above.

$L_2$ . ANQI remains on the high-throughput side of all four baselines throughout the high-recall region on SIFT1M and DEEP1M. Near recall 0.95, it is 2.57 to 2.84 $\times$  faster than HRNN on SIFT1M and 2.14 to 2.34 $\times$  faster on DEEP1M. The separation persists as  $k$  increases, even though every method must return larger reverse-neighbor sets.

The distance counts explain this result. Across the nine  $L_2$  operating points, ANQI performs 2.6k to 15.5k complete online distance evaluations per query, versus 5.74k to 31.7k for HRNN. This is a 2.0 to 2.6 $\times$  reduction. The displayed HAMG, SFT, and RDT endpoints

require 161k to 107M evaluations and often still miss the recall target. ANQI still verifies its candidates; the advantage comes from making the high-recall candidate pool cheaper to obtain.

GIST1M shows the largest throughput gap. ANQI is 3.9 to 4.4 $\times$  faster than HRNN at the selected points, while HAMG misses recall 0.95 for all three ranks and SFT/RDT fall to single-digit QPS. At  $k = 10$ , ANQI reaches recall 0.96 and precision 0.997 at 4,400 QPS. Increasing the search budget reaches recall 0.996 at 220 QPS without reducing precision. The unchanged precision confirms that the added recall does not come from additional false positives.

*Inner product.* The IP panels compare only native reverse-MIPS methods. ANQI traces a continuous approximate frontier, whereas SAH and the SIMPFER variants produce isolated, much slower points. At the recall-0.99 target, ANQI reaches recall 0.99 to 1.0 with precision 1.0 at 510 to 23k QPS. The specialized methods reach recall 1.0, but their precision ranges from 0.56 to 1.0 and their throughput from 0.50 to 50 QPS.

For IP, ANQI fits the compact representation directly to the exact signed threshold matrix  $T[o, k] = -\text{ip}_k(o)$ ; rank-specific residuals retain object- and rank-specific boundary detail.

The advantage is present on both IP datasets. On Music100, ANQI is 650 to 2,900 $\times$  faster than the fastest qualifying baseline across the three ranks. On MSMarcoWebSearch1M, the speedup grows from 10 $\times$  at  $k = 10$  to 160 $\times$  at  $k = 100$ , with precision 1.0 at every selected point. Reporting precision beside recall matters here: several exact-recall baselines admit many false positives, especially on MSMarcoWebSearch1M.

Across all target ranks and both  $L_2$  and IP, ANQI shifts the high-recall frontier by reducing candidate work before verification. Precision remains at least 0.997 at every selected ANQI point.

## 6.3 Cost and breakdown

Higher query throughput matters only if construction and storage remain manageable. The trade-off depends on the baseline. Against HRNN, ANQI builds 1.5 to 1.7 $\times$  faster on the three  $L_2$  datasets and uses 2.4 to 9.0 $\times$  less index space. Its bundle includes the lifted graph, verifier, shared basis, and codebook, and the same bundle serves all three target ranks.

The plain HNSW shared by SFT and RDT is faster to build and slightly smaller than ANQI. It stores no RkNN-specific ranked-neighbor lists, reverse lists, or learned threshold state; those methods instead expand and verify candidates online. Their displayed endpoints evaluate one to four orders of magnitude more distances than ANQI and often miss the GIST1M target.

The IP baselines make the opposite trade-off: their indexes build in 1.5 to 130 seconds, well below ANQI’s 550 to 880 seconds, but their query throughput is lower by factors of 10 to 2,900 at the selected points. ANQI thus pays a larger one-time build cost on IP in exchange for higher query throughput.

Table 6 identifies the remaining bottlenecks. The base-space AKNN warm start consumes 39.0 seconds, or 74% of post-training construction; graph build and save consume another 12.1 seconds, or 23%. Lift preparation, verifier encoding, and other setup together account for only 3.1%. The one-time 36-second model fit is a separate 41% of the 89-second total in table 5. Construction optimization

**Table 6: DEEP1M construction and matched-quality query-time breakdown.**

| Construction: 52.8 s |          |       | Query at matched quality |        |        |     |               |                |  |
|----------------------|----------|-------|--------------------------|--------|--------|-----|---------------|----------------|--|
| Component            | Time (s) | Share | $k$                      | Target | Recall | QPS | Graph $\mu$ s | Verify $\mu$ s |  |
| Base-space AKNN      | 39.0     | 74%   | 10                       | .95    | .96    | 78k | 790           | 30             |  |
| Graph build/save     | 12.1     | 23%   | 10                       | .99    | .99    | 33k | 1,900         | 33             |  |
| Lift preparation     | 0.74     | 1.4%  | 50                       | .95    | .96    | 62k | 1,000         | 43             |  |
| LRQ encode/save      | 0.25     | 0.50% | 50                       | .99    | .99    | 33k | 2,200         | 51             |  |
| Other setup          | 0.67     | 1.3%  | 100                      | .95    | .97    | 45k | 1,400         | 60             |  |
|                      |          |       | 100                      | .99    | 1.00   | 18k | 3,400         | 66             |  |

**Table 7: Representative SIFT points near recall 0.95. Q/D is QPS/mean complete online distance evaluations per query; B/I is query-index build time/index GB ( $10^9$  bytes). The 1B R/P row uses the audited proxy protocol in fig. 5.**

| Scale | Method | R/P      | Q/D      | B/I       |
|-------|--------|----------|----------|-----------|
| 10M   | ANQI   | .96/1.00 | 29k/4.0k | 160s/8.4  |
|       | HRNN   | .95/1.00 | 13k/9.0k | 2,600s/87 |
| 100M  | ANQI   | .95/1.00 | 12k/10k  | 2,900s/84 |
| 1B    | ANQI   | .95/95   | 11k/1.0k | 1.3h/110  |

should therefore focus on model fitting and the forward warm start; the lift and residual-encoding pass are already minor costs.

The query breakdown points to the same conclusion. Raising either  $k$  or the recall target increases graph time from 790 to 3,400  $\mu$ s, while verification stays between 30 and 66  $\mu$ s and transformation stays between 0.40 and 0.51  $\mu$ s. Graph traversal consequently accounts for 96 to 98% of query CPU time; verification accounts for only 1.7 to 4.1%.

ANQI requires more offline work than the lightweight HNSW and IP baselines, but less than HRNN. Online verification is not the query bottleneck, and verifier encoding is negligible after training. Model fitting and neighbor-graph construction remain the main opportunities for lower build cost.

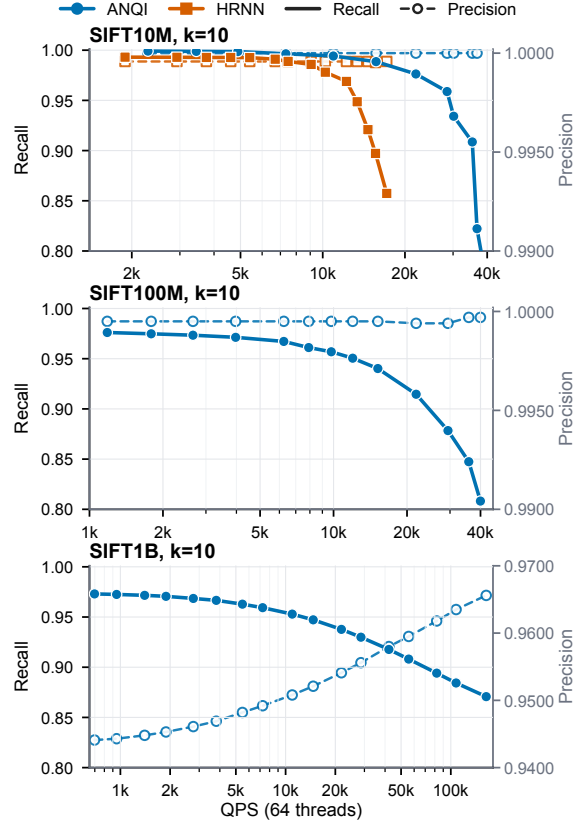
#### 6.4 Scalability from 10M to 1B vectors

Million-vector gains are insufficient if they disappear as the collection grows. We therefore extend three orders of magnitude beyond the main benchmark, with a 10M-to-1B sweep spanning the final two. At recall near 0.95, ANQI processes 28,520 QPS at 10M vectors, 11,976 QPS at 100M, and 10,964 QPS at 1B.

At 10M, ANQI reaches recall/precision 0.96/1.00 at 29k QPS, versus 0.95/1.00 at 13k QPS for HRNN. This 2.1 $\times$  throughput gain comes with 4.0k rather than 9.0k online distance evaluations per query. ANQI also builds 16 $\times$  faster and uses a 10 $\times$  smaller index. The query advantage therefore accompanies lower construction and storage cost at this scale.

At 100M, ANQI sustains recall 0.95 and precision 1.00 at 12k QPS with an 84 GB index. HRNN is omitted because its measured 10M artifact projects an 810 GiB index at 100M, beyond the available 88 GiB disk. Since it already fails this storage gate at 100M, we do not attempt it at 1B.

In the 1B scaling prototype, recall rises from 0.87 to 0.97 while precision changes from 0.97 to 0.94. Its 0.95/0.95 point reaches 11k QPS with 1.0k distance evaluations per query. Starting from the audited forward top-10 proxy, the 110 GB query index builds in 1.3



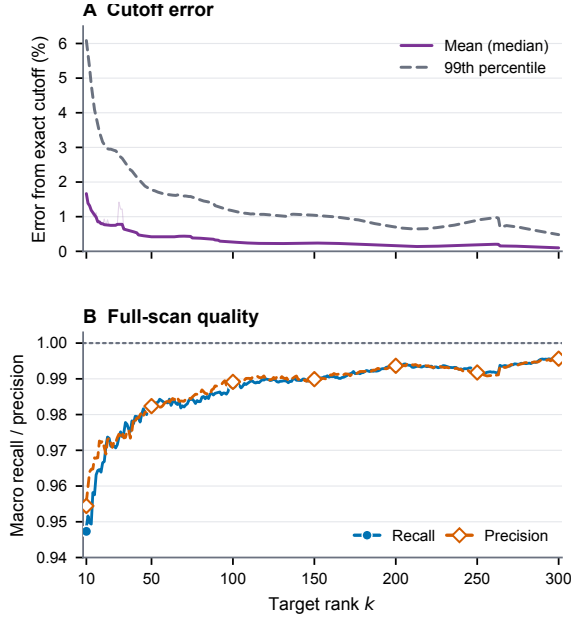
**Figure 5: SIFT reverse-10NN scalability from 10M to 1B vectors. Filled and hollow traces show recall and precision; panels use independent QPS ranges. HRNN appears only at 10M. The 1B panel uses the earlier M6/Vamana scale prototype and an audited forward-derived proxy table.**

hours. Against 6,000 exact sampled base queries, the proxy attains ID recall@10 and tie-aware recall@10 of 0.99.

Under this audited proxy protocol, the 1B prototype maintains recall and precision near 0.95. At 10M, ANQI also improves construction time and index size relative to HRNN.

#### 6.5 Local Radius Law in Euclidean Space

This Euclidean experiment tests the compact rank structure suggested by the Local Radius Law on exact squared-distance thresholds. We isolate the low-rank term by setting  $\epsilon_{o,k} = 0$  in eq. (21). A standalone rank-8 model learns a shared mean and PCA basis from 120,000 rows of the exact DEEP1M squared- $L_2$  threshold matrix over ranks 1,  $\dots$ , 500. Each of the one million objects is represented by eight coefficients computed from its complete 500-rank row. Because every evaluated rank  $k = 10, \dots, 300$  participates in this representation, the experiment tests target-aware in-range compression rather than unseen-rank extrapolation. Panel A measures threshold error over all base objects; relative ratios exclude exact-zero thresholds, which account for at most 0.012%. Panel B applies the reconstructed cutoff to a full scan, removing graph-search misses from the diagnosis.



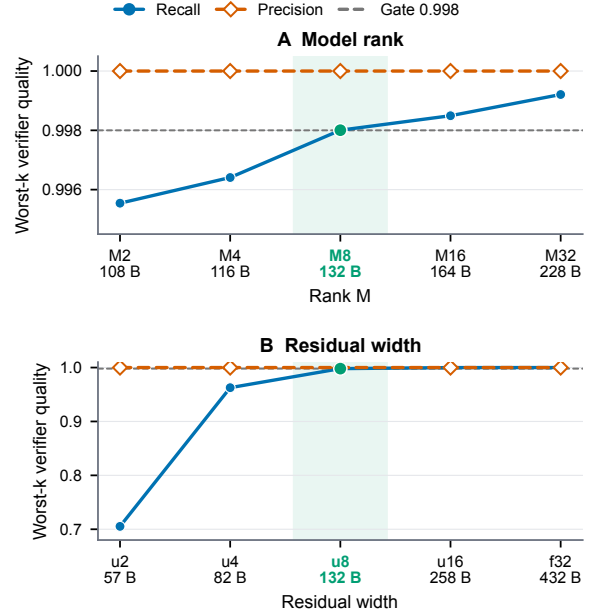
**Figure 6: Standalone M8 Euclidean threshold modeling on DEEP1M for every integer  $k = 10, \dots, 300$ . Panel A reports mean and 99th-percentile cutoff error over all one million objects. The faint mean-error trace is raw; the bold curve is a centered 7-rank median used only as a visual guide, and reported values remain unsmoothed. Panel B applies the same cutoffs in a full-base scan with no graph, search budget, or QPS condition; the horizontal line marks exact quality (1.0). Lines use 1,000 fixed queries, while large symbols use all 10,000 queries at seven ranks.**

Figure 6 shows that eight coefficients capture most of each object’s threshold curve even without residual correction. Mean absolute relative error is 1.7% at  $k = 10$  and 0.10% at  $k = 300$ ; the corresponding 99th-percentile errors are 6.1% and 0.48%. Across the full rank range, 49 to 51% of reconstructions underestimate the exact threshold, so the unconstrained low-rank term has nearly balanced two-sided error. On all-query anchors, macro recall/precision is 0.95/0.95 at  $k = 10$  and 1.00/1.00 at  $k = 300$ ; every anchor at  $k \geq 50$  exceeds 0.98 on both measures. Thus the shared rank-8 model explains most of the threshold structure across ranks with 32 bytes per vector, but leaves measurable boundary error at small  $k$ .

Eight coefficients capture sufficient rank structure to exceed 0.98 recall and precision at every all-query anchor with  $k \geq 50$ . The remaining error, most visible at  $k = 10$ , motivates the residual correction evaluated next.

## 6.6 Why ANQI works: lifting and verifier ablations

Three ablations isolate graph geometry, verifier capacity, and the construction pivot while holding the remaining components fixed.



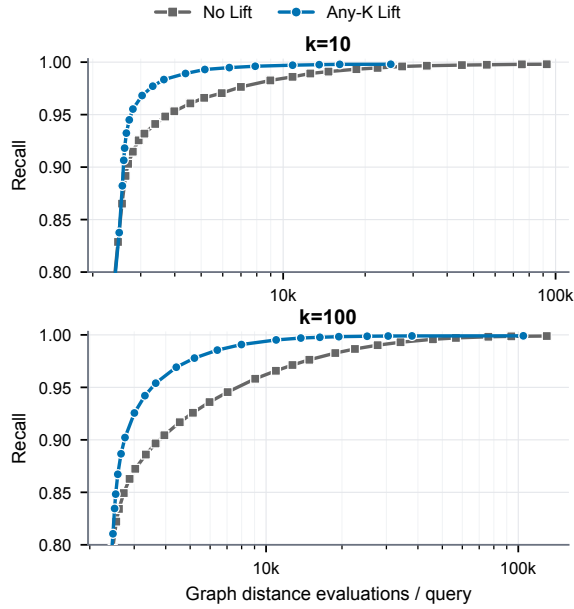
**Figure 7: Compact-verifier ablation on DEEP1M. Quality is the worst macro recall and precision over  $k = 10, 50, 100$  in full-base verification. Panel A fixes  $u8$  and varies  $M$ ; Panel B fixes  $M = 8$  and varies residual width. Tick labels include total verifier storage. Green marks  $M8+u8$ , the lowest-storage configuration meeting the recall-0.998/precision-0.999 selection criterion at 132 B/vector.**

*Graph geometry.* Figure 8 changes only graph geometry. At the first measured point reaching recall 0.95, Lift reduces graph-distance work by factors of 1.4 $\times$  and 2.5 $\times$  for  $k = 10$  and 100. At recall 0.99, the reductions grow to 2.8 $\times$  and 3.5 $\times$ . The leftward separation is therefore strongest in the high-recall tail and increases with  $k$ .

Both arms use  $M8+u8$  with precision 1.0. The EXACT-F32 control has identical candidates, graph work, and candidate recall; maximum M8 recall gaps are 0.002 and 0.001 for the displayed ranks. The matched controls isolate the high-recall gain to navigation. Lift adds 1.2 seconds of build time (49 to 50 seconds) and about 4 MB (0.71 to 0.72 GB).

*Construction pivot.* We vary the construction pivot while retaining target- $k$  candidate collection. Figure 9 shows that every tested pivot reaches recall 0.99 for all three query ranks, including  $K_{\text{pivot}} = 25$  at  $k = 100$ . Within this evaluated range, the target- $k$  envelope remains effective above the construction pivot. At  $k = 10$ , the four curves nearly overlap. At  $k = 50$  and  $L = 240$ , all pivots are essentially at the 0.99 target; the  $K_{\text{pivot}} = 25$  run crosses at the next sampled budget,  $L = 480$ . At  $k = 100$ , all four pivots cross at  $L = 480$  with about 11k graph-distance evaluations. Candidate counts remain target-controlled at approximately 11, 50, and 100 for  $k = 10, 50, 100$ .

The fixed-G48 construction cost is likewise insensitive to the pivot: 66 to 69 seconds and about 0.60 GB across the sweep. We



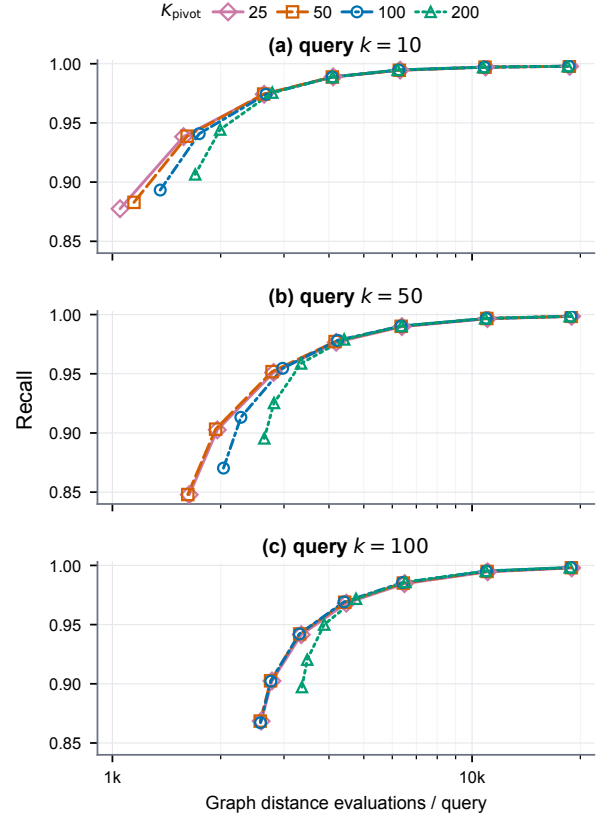
**Figure 8: Controlled DEEP1M graph-geometry ablation for  $k = 10$  and  $100$ .** Panels plot recall against graph distance evaluations per query. No Lift uses the original 96-dimensional graph; construction-rank Lift uses the exact- $r_{100}$  98-dimensional graph. Both arms otherwise share exact thresholds, degree-48 build and search, and the M8+u8 verifier. Precision is 1.0 throughout.

retain  $K_{\text{pivot}} = 100$  because it aligns with the materialized  $k \leq 100$  range.

*Verifier capacity and residual width.* Figure 7 reports controlled slices through the complete  $M \in \{2, 4, 8, 16, 32\}$  by residual-width grid. With u8 fixed, worst- $k$  recall rises from 0.996 at M2 to 0.996 at M4 and 0.998 at M8. Increasing the rank further reaches only 0.999 at M16 and 0.999 at M32, while storage grows from 132 B/vector at M8 to 164 and 228 B/vector.

With  $M = 8$  fixed, two- and four-bit residuals are too coarse: their conservative floors retain precision 1.0 but reach recall only 0.71 and 0.96. U8 meets the high-fidelity selection criterion at 0.998/1.0 using 132 B/vector. U16 is nearly exact at 1.00/1.0 but costs 258 B/vector, and a float32 residual reaches 1.0/1.0 at 432 B/vector. M8+u8 therefore uses about one third of the 400 B/vector source-threshold table while preserving high-fidelity verification. Candidate generation and distance counts are invariant across verifier arms, so this ablation establishes a storage-quality trade-off rather than a query-speed advantage.

In matched graph construction, the lift accounts for the observed reduction in navigation work, while the verifier controls predicate accuracy. Every tested pivot from 25 through 200 reaches recall 0.99 for  $k = 10, 50, 100$ , including cross-pivot queries in the evaluated range. M8+u8 meets the verifier selection criterion at 132 B/vector, and larger models add only small quality gains.



**Figure 9: Construction-pivot robustness on DEEP1M for all three target ranks.** We build one G48 Lift graph for each  $K_{\text{pivot}} \in \{25, 50, 100, 200\}$ , and every graph serves  $k \in \{10, 50, 100\}$  with the target- $k$  M8+u8 verifier envelope. Candidate collection uses the target- $k$  envelope, and measured candidate counts follow the target rank. Precision is 1.0 throughout.

## 7 Conclusion

Across five million-scale datasets, ANQI delivers multi-fold throughput gains over recall-qualified baselines at high-recall operating points (recall 0.95 for  $L_2$  and 0.99 for IP), while maintaining precision of at least 0.99. To our precision of at least 0.99. The same lifted graph and compact verifier serve all materialized target ranks without rebuilding the index: the Local Radius Law gives the Euclidean rationale for shared rank structure, while IP uses the same layout fitted directly to signed thresholds. This separation keeps candidate coverage in the graph and target-rank fidelity in the verifier.

To our knowledge, ANQI is the first  $RkNN$  index to demonstrate both construction and query processing at billion-vector scale: its SIFT1B prototype achieves recall and precision above 0.95 at more than 10,000 QPS. These results show that reverse candidate routing can reuse forward ANN infrastructure while preserving compact, object- and rank-specific decision thresholds.

## References

- [1] Elke Achtert, Christian Böhm, Peer Kröger, Peter Kunath, Alexey Pryakhin, and Matthias Renz. 2006. Approximate Reverse  $k$ -Nearest Neighbor Queries in General Metric Spaces. In *CIKM*. 788–789. doi:10.1145/1183614.1183731
- [2] Elke Achtert, Christian Böhm, Peer Kröger, Peter Kunath, Alexey Pryakhin, and Matthias Renz. 2006. Efficient Reverse  $k$ -Nearest Neighbor Search in Arbitrary Metric Spaces. In *SIGMOD*. doi:10.1145/1142473.1142531
- [3] Daichi Amagata and Takahiro Hara. 2021. Reverse maximum inner product search: How to efficiently find users who would like to buy my item. In *RecSys*. 273–281. doi:10.1145/3460231.3474229
- [4] Daichi Amagata and Takahiro Hara. 2023. Reverse Maximum Inner Product Search: Formulation, Algorithms, and Analysis. *ACM Transactions on the Web* 17, 4, Article 26 (2023), 23 pages. doi:10.1145/3587215
- [5] David Arthur and Steve Y. Oudot. 2010. Reverse Nearest Neighbors Search in High Dimensions Using Locality-Sensitive Hashing. *arXiv preprint arXiv:1011.4955* (2010).
- [6] Franz Aurenhammer. 1987. Power Diagrams: Properties, Algorithms and Applications. *SIAM J. Comput.* 16, 1 (1987), 78–96. doi:10.1137/0216006
- [7] Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nice, and Ulrich Paquet. 2014. Speeding up the Xbox recommender system using a euclidean transformation for inner-product spaces. In *RecSys*. 257–264. doi:10.1145/2645710.2645741
- [8] Zhengyang Bai, Peng Chen, and Mohamed Wahib. 2026. RT-RkNN: Reverse  $k$  Nearest Neighbor Queries as a Graphics Ray Casting Problem. *arXiv preprint arXiv:2605.26671* (2026).
- [9] Max Berrendorf, Felix Borutta, and Peer Kröger. 2019.  $k$ -Distance Approximation for Memory-Efficient RkNN Retrieval. In *SISAP (LNCS, Vol. 11807)*. 57–71. doi:10.1007/978-3-030-32047-8\_6
- [10] Guillaume Casanova, Elias Englmeier, Michael E. Houle, Peer Kröger, Michael Nett, Erich Schubert, and Arthur Zimek. 2017. Dimensional Testing for Reverse  $k$ -Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 10, 7 (2017), 769–780. doi:10.14778/3067421.3067426
- [11] Tingyang Chen, Cong Fu, Xiangyu Ke, Yunjun Gao, Yabo Ni, and Anxiang Zeng. 2025. Stitching Inner Product and Euclidean Metrics for Topology-Aware Maximum Inner Product Search. In *SIGIR*. 2341–2350. doi:10.1145/3726302.3730088
- [12] Tingyang Chen, Cong Fu, Kun Wang, Xiangyu Ke, Yunjun Gao, Wenchao Zhou, Yabo Ni, and Anxiang Zeng. 2025. Maximum Inner Product is Query-Scaled Nearest Neighbor. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1770–1783. doi:10.14778/3725688.3725705
- [13] Yongjian Chen, Tao Guan, and Cheng Wang. 2010. Approximate Nearest Neighbor Search by Residual Vector Quantization. *Sensors* 10, 12 (2010), 11259–11273. doi:10.3390/s101211259
- [14] Otfried Cheong, Antoine Vigneron, and Juyoung Yon. 2011. Reverse Nearest Neighbor Queries in Fixed Dimension. *International Journal of Computational Geometry & Applications* 21, 2 (2011), 179–188. doi:10.1142/S0218195911003607
- [15] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient  $k$ -Nearest Neighbor Graph Construction for Generic Similarity Measures. In *WWW*. 577–586. doi:10.1145/1963405.1963487
- [16] Carl Eckart and Gale Young. 1936. The Approximation of One Matrix by Another of Lower Rank. *Psychometrika* 1, 3 (1936), 211–218. doi:10.1007/BF02288367
- [17] Cong Fu and Deng Cai. 2016. EFANNA: An Extremely Fast Approximate Nearest Neighbor Search Algorithm Based on  $k$ NN Graph. *arXiv preprint arXiv:1609.07228* (2016).
- [18] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474. doi:10.14778/3303753.3303754
- [19] Yujian Fu, Cheng Chen, Xiaohui Chen, Weng-Fai Wong, and Bingsheng He. 2024. Optimizing the Number of Clusters for Billion-Scale Quantization-Based Nearest Neighbor Search. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 6786–6800. doi:10.1109/TKDE.2024.3408815
- [20] Yujian Fu, Cheng Chen, Yao Chen, Weng-Fai Wong, and Bingsheng He. 2025. Vista: Vector Indexing and Search for Large-Scale Imbalanced Datasets. In *ICDE*. IEEE, Hong Kong, 543–556. doi:10.1109/ICDE65448.2025.00047
- [21] Yujian Fu, Cheng Chen, Yao Chen, Weng-Fai Wong, and Bingsheng He. 2026. Geometric Partition for Billion-Scale Approximate Nearest Neighbor Search. *IEEE Transactions on Knowledge and Data Engineering* 38, 5 (2026), 2865–2879. doi:10.1109/TKDE.2026.3667997
- [22] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *CVPR*. 2946–2953.
- [23] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *ICML (Proceedings of Machine Learning Research, Vol. 119)*. 3887–3896.
- [24] Qiang Huang, Yanhao Wang, and Anthony K. H. Tung. 2023. SAH: Shifting-Aware Asymmetric Hashing for Reverse  $k$ -Maximum Inner Product Search. In *AAAI*. 4312–4321. doi:10.1609/aaai.v37i14.25550
- [25] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57
- [26] Flip Korn and S. Muthukrishnan. 2000. Influence Sets Based on Reverse Nearest Neighbor Queries. In *SIGMOD*. 201–212. doi:10.1145/335191.335415
- [27] Elizaveta Levina and Peter J. Bickel. 2004. Maximum Likelihood Estimation of Intrinsic Dimension. In *Advances in Neural Information Processing Systems*, Vol. 17. MIT Press, 777–784.
- [28] Jie Liu, Xiao Yan, Xinyan Dai, Zhirong Li, James Cheng, and Ming-Chang Yang. 2020. Understanding and Improving Proximity Graph Based Maximum Inner Product Search. In *AAAI*. 139–146. doi:10.1609/aaai.v34i01.5344
- [29] D. O. Loftsgaarden and C. P. Quesenberry. 1965. A Nonparametric Estimate of a Multivariate Density Function. *The Annals of Mathematical Statistics* 36, 3 (1965), 1049–1051. doi:10.1214/aoms/1177700079
- [30] Yu A. Malkov and Dmitry A. Yashunin. 2020. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [31] Magdalen Dobson Manohar, Taekseung Kim, and Guy E. Blelloch. 2025. Range Retrieval with Graph-Based Indices. *arXiv preprint arXiv:2502.13245* (2025).
- [32] Magdalen Dobson Manohar, Zheqi Shen, Guy E. Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2024. ParlayANN: Scalable and Deterministic Parallel Graph-Based Approximate Nearest Neighbor Search Algorithms. In *PPoPP*. 270–285. doi:10.1145/3627535.3638475
- [33] Stanislav Morozov and Artem Babenko. 2018. Non-metric similarity graphs for maximum inner product search. In *Advances in Neural Information Processing Systems*, Vol. 31. Curran Associates, Inc., 4726–4735.
- [34] Sandra Obermeier, Max Berrendorf, and Peer Kröger. 2020. Memory-Efficient RkNN Retrieval by Nonlinear  $k$ -Distance Approximation. In *WI-IAT*. 387–390.
- [35] Amit Singh, Hakan Ferhatoşmanoglu, and Ali Şaman Tosun. 2003. High Dimensional Reverse Nearest Neighbor Queries. In *CIKM*. 91–98. doi:10.1145/956863.956882
- [36] Yitong Song, Kai Wang, Bin Yao, Zhida Chen, Jiong Xie, and Feifei Li. 2024. Efficient Reverse  $k$  Approximate Nearest Neighbor Search Over High-Dimensional Vectors. In *ICDE*. 4262–4274. doi:10.1109/ICDE60146.2024.00325
- [37] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., 13748–13758.
- [38] Shulong Tan, Zhaozhuo Xu, Weijie Zhao, Hongliang Fei, Zhixin Zhou, and Ping Li. 2021. Norm Adjusted Proximity Graph for Fast Inner Product Retrieval. In *KDD*. 1552–1560. doi:10.1145/3447548.3467412
- [39] Yufei Tao, Dimitris Papadias, Xiang Lian, and Xiaokui Xiao. 2007. Multidimensional Reverse  $k$ NN Search. *The VLDB Journal* 16, 3 (2007), 293–316. doi:10.1007/s00778-005-0168-2
- [40] Wenxuan Xia, Mingyu Yang, Wentao Li, and Wei Wang. 2026. HRNN: A Hybrid Graph Index for Approximate Reverse  $k$ -Nearest Neighbor Search on High-Dimensional Vectors. *arXiv preprint arXiv:2606.03225* (2026).
- [41] Xi Zhao, Bolong Zheng, Xiaomeng Yi, Xiaofan Luan, Charles Xie, Xiaofang Zhou, and Christian S. Jensen. 2023. FARGO: Fast Maximum Inner Product Search via Global Multi-Probing. *Proceedings of the VLDB Endowment* 16, 5 (2023), 1100–1112. doi:10.14778/3579075.3579084
- [42] Zhixin Zhou, Shulong Tan, Zhaozhuo Xu, and Ping Li. 2019. Möbius transformation for fast inner product search on graph. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., 8216–8227.

## A Proofs and supplementary analysis

This appendix gives the complete algebra behind the statements in Sections 4 and 5. We use exact thresholds and real arithmetic unless stated otherwise. Thus the proofs separate identities that hold for the represented predicate from the graph-coverage property, which is evaluated empirically by the finite ANN traversal.

### A.1 Exact lifting and rank sharing

The first result establishes the role of lifting in the system. It does not change which objects satisfy the reverse predicate. It changes only the representation so that a forward Euclidean search order can find those objects, after which the same predicate can be applied to the retrieved candidates.

**PROOF OF THEOREM 1.** We first verify the predicate in the original score space. For  $L_2$ , (10) gives

$$\begin{aligned} \|q - o\|^2 \leq r_k(o)^2 &\iff \|q\|^2 + \|o\|^2 - 2 \cdot \langle q, o \rangle \leq r_k(o)^2 \\ &\iff \langle q, o \rangle - \frac{1}{2} \cdot (\|o\|^2 - r_k(o)^2) \geq \frac{1}{2} \cdot \|q\|^2. \end{aligned}$$

The left-hand side is exactly  $s_k(q, o) = \langle q^\ell, o_k^\ell \rangle$  for the  $L_2$  row in (12), and the right-hand side is  $\tau(q)$ . For inner product, the same lifted inner product is  $s_k(q, o) = \langle q, o \rangle - \text{ip}_k(o)$ , so

$$o \in A_k(q) \iff s_k(q, o) \geq 0 = \tau(q).$$

Thus, for either metric,

$$o \in A_k(q) \iff s_k(q, o) \geq \tau(q).$$

Now consider the norm-equalized vectors in (14). The last coordinate is real because  $M_\star$  is at least the largest norm of the lifted indexed vectors. Hence  $\|\bar{o}_k\|^2 = \|o_k^\ell\|^2 + M_\star^2 - \|o_k^\ell\|^2 = M_\star^2$ . Using  $\bar{q} = [q^\ell, 0]$  and expanding the squared distance gives

$$\begin{aligned} \|\bar{q} - \bar{o}_k\|^2 &= \|\bar{q}\|^2 + \|\bar{o}_k\|^2 - 2 \cdot \langle \bar{q}, \bar{o}_k \rangle \\ &= \|q^\ell\|^2 + M_\star^2 - 2 \cdot \langle q^\ell, o_k^\ell \rangle \\ &= \|q^\ell\|^2 + M_\star^2 - 2 \cdot s_k(q, o). \end{aligned}$$

Since the first two terms are independent of  $o$ , a larger score is equivalent to a smaller Euclidean distance. Substituting  $s_k(q, o) \geq \tau(q)$  into this identity yields

$$\begin{aligned} s_k(q, o) \geq \tau(q) &\iff \|\bar{q} - \bar{o}_k\|^2 \leq \|q^\ell\|^2 + M_\star^2 - 2 \cdot \tau(q) \\ &\iff \|\bar{q} - \bar{o}_k\|^2 \leq \rho_k(q)^2, \end{aligned}$$

which proves the lifted-ball equivalence and the score-ordering claim. Finally, for any candidate set  $C(q)$ , applying this pointwise equivalence only to vectors in  $C(q)$  gives

$$\{o \in C(q) : \|\bar{q} - \bar{o}_k\|^2 \leq \rho_k(q)^2\} = C(q) \cap A_k(q).$$

This is a statement about filtering the supplied candidates; it does not claim that a finite graph traversal necessarily supplies every member of  $A_k(q)$ .  $\square$

**PROOF OF PROPOSITION 1.** For the  $L_2$  case, define  $N_o(r) = |\{x \in \mathcal{X} \setminus \{o\} : \|x - o\| \leq r\}|$ . By definition,  $r_k(o)$  is a  $k$ th-neighbor radius, so  $N_o(r_k(o)) \geq k$ . If  $K \geq k$ , the  $K$ th order statistic cannot be smaller than the  $k$ th one:

$$r_k(o) \leq r_K(o).$$

Therefore, whenever  $\|q - o\| \leq r_k(o)$ , we also have  $\|q - o\| \leq r_K(o)$ , proving  $A_k(q) \subseteq A_K(q)$ .

For inner product, let the scores  $\{\langle x, o \rangle : x \neq o\}$  be sorted in nonincreasing order. The  $k$ th order statistic satisfies  $\text{ip}_k(o) \geq$

$\text{ip}_K(o)$  for  $K \geq k$ . Hence

$$\langle q, o \rangle \geq \text{ip}_k(o) \implies \langle q, o \rangle \geq \text{ip}_K(o),$$

which proves the same set containment. Theorem 1 maps each of these pointwise predicates to its lifted-ball predicate, so the construction-rank acceptance region at  $K$  is a valid exact candidate envelope for every represented target rank  $k \leq K$ . The proposition does not by itself provide a finite-beam graph-coverage guarantee.  $\square$

The shared-rank result is therefore a candidate-generation statement: a graph built for a larger construction rank contains the exact acceptance region of every smaller represented rank. It says nothing about how well a finite-beam traversal covers that region; that is the quantity measured in the cross-pivot experiments.

**PROOF OF LEMMA 1.** By definition of the construction-rank score,

$$s_{K_{\text{pivot}}}(q, v) = \langle q, v \rangle - \frac{1}{2} \cdot (\|v\|^2 - t_{K_{\text{pivot}}}(v)).$$

Adding the correction and grouping the two terms that contain  $t_{K_{\text{pivot}}}(v)$  gives

$$\begin{aligned} s_{K_{\text{pivot}}}(q, v) &+ \frac{1}{2} \cdot (\tilde{t}_k(v) - t_{K_{\text{pivot}}}(v)) \\ &= \langle q, v \rangle - \frac{1}{2} \cdot \|v\|^2 + \frac{1}{2} \cdot t_{K_{\text{pivot}}}(v) + \frac{1}{2} \cdot \tilde{t}_k(v) - \frac{1}{2} \cdot t_{K_{\text{pivot}}}(v) \\ &= \langle q, v \rangle - \frac{1}{2} \cdot (\|v\|^2 - \tilde{t}_k(v)) \\ &= \tilde{s}_{\tilde{t}_k}(q, v). \end{aligned}$$

The identity is algebraic and holds for any represented rank, including a rank larger than  $K_{\text{pivot}}$ . If the reconstruction is exact,  $\tilde{t}_k(v) = t_k(v)$ , then the final expression is the exact target- $k$  reverse-aware score by (10). What remains empirical beyond the construction pivot is whether the graph traversal reaches all objects in the new target-rank envelope, not whether this score correction is algebraically valid.  $\square$

### A.2 Euclidean local-radius analysis

The next derivation explains why neighboring Euclidean ranks can share a compact model. Under a locally stable density and intrinsic dimension, the squared radius is approximately a power of the rank. Expanding the small deviation from a reference exponent produces shared rank functions and object-specific coefficients, which is the structural motivation for the low-rank verifier.

**PROOF OF PROPOSITION 2.** Under the local ball-mass model,

$$N_o(r) = \rho(o) \cdot V_{d(o)} \cdot r^{d(o)}.$$

At the  $k$ th neighbor radius,  $N_o(r_k(o)) = k$ , so

$$r_k(o)^{d(o)} = \frac{k}{\rho(o) \cdot V_{d(o)}}.$$

Raising both sides to the power  $2/d(o)$  gives

$$\begin{aligned} r_k(o)^2 &= \left( \frac{k}{\rho(o) \cdot V_{d(o)}} \right)^{2/d(o)} \\ &= [\rho(o) \cdot V_{d(o)}]^{-2/d(o)} \cdot k^{2/d(o)} \\ &= \gamma(o) \cdot k^{v(o)}. \end{aligned}$$

with  $v(o) = 2/d(o)$  and  $\gamma(o) = [\rho(o) \cdot V_{d(o)}]^{-v(o)}$ .

Write  $v(o) = v_0 + \beta(o)$ . Then

$$k^{v(o)} = k^{v_0 + \beta(o)} = k^{v_0} \cdot \exp(\beta(o) \cdot \ln k).$$

For  $x = \beta(o) \cdot \ln k$ , Taylor's theorem gives

$$e^x = \sum_{m=0}^p \frac{x^m}{m!} + R_p(x), \quad |R_p(x)| \leq e^{|x|} \frac{|x|^{p+1}}{(p+1)!}.$$

Multiplying by  $\gamma(o) \cdot k^{v_0}$  and inserting  $x = \beta(o) \cdot \ln k$  produces

$$r_k(o)^2 = \sum_{m=0}^{\infty} \underbrace{\frac{\gamma(o) \cdot \beta(o)^m}{m!}}_{f_m(o)} \underbrace{k^{v_0} \cdot (\ln k)^m}_{\phi_m(k)}.$$

The same multiplication applied to the Taylor remainder gives

$$|R_p(o, k)| \leq \gamma(o) \cdot k^{v_0} \cdot e^{|\beta(o) \cdot \ln k|} \cdot \frac{|\beta(o) \cdot \ln k|^{p+1}}{(p+1)!},$$

which is (25). The factorization shows why a small shared rank basis can be useful when  $|\beta(o) \cdot \ln k|$  is moderate. It is an object-local approximation: changes in density, curvature, or intrinsic dimension can invalidate one exponent over a wider rank interval, and no claim about the ambient dimension  $D$  is needed for the derivation.  $\square$

PROOF OF COROLLARY 1. Applying Proposition 2 at  $k$  and  $K$  gives

$$r_k(o)^2 = \gamma(o) \cdot k^{v(o)}, \quad r_K(o)^2 = \gamma(o) \cdot K^{v(o)}.$$

Because  $\gamma(o) > 0$ , division cancels the object-specific intensity and volume factor:

$$\frac{r_k(o)^2}{r_K(o)^2} = \left(\frac{k}{K}\right)^{v(o)}.$$

The local mass equation also gives  $N_o(r_k(o)) = k$  and  $N_o(r_K(o)) = K$ . The shell between the two radii therefore has mass

$$N_o(r_k(o)) - N_o(r_K(o)) = k - K.$$

The result describes the local model's expected shell mass; it does not assert that a finite sampled dataset has exactly uniform shell placement at every rank.  $\square$

### A.3 IP quantiles and one-sided verification

The IP result has a different interpretation. Inner-product thresholds are directional projection quantiles, not Euclidean radii. The identity below explains the exact threshold being modeled, while the low-rank-plus-LRQ representation remains an empirical compression of that signed threshold matrix.

PROOF OF PROPOSITION 3. Suppose first that  $o \neq 0$  and define  $u_o = o/\|o\|$ . For every base vector  $x$ , bilinearity of the inner product gives

$$\langle x, o \rangle = \langle x, \|o\|u_o \rangle = \|o\| \cdot \langle x, u_o \rangle.$$

The multiplier  $\|o\|$  is positive and is the same for every  $x$ , so it preserves the ordering of all projection scores. Consequently, the  $k$ th largest inner product equals the same positive multiplier times the  $k$ th largest projection:

$$\text{ip}_k(o) = \|o\| \cdot \pi_k(o).$$

The reverse predicate becomes

$$\begin{aligned} \langle q, o \rangle \geq \text{ip}_k(o) &\iff \|o\| \cdot \langle q, u_o \rangle \geq \|o\| \cdot \pi_k(o) \\ &\iff \langle q, u_o \rangle \geq \pi_k(o), \end{aligned}$$

where the second equivalence divides by the positive number  $\|o\|$ . Since  $T[o, k] = -\text{ip}_k(o)$  by definition, the signed threshold row is  $T[o, k] = -\|o\| \cdot \pi_k(o)$ . If  $o = 0$ , every inner product with  $o$  is zero;

all order statistics and the signed threshold row are therefore zero, which is the stated special case.  $\square$

PROOF OF PROPOSITION 4. Assume first that the reconstruction is restrictive, so  $\hat{t}_k(o) \leq t_k(o)$  for every  $o$ . If an object is accepted by the represented verifier, then

$$\kappa(q, o) \leq \hat{t}_k(o) \leq t_k(o),$$

and it is therefore also in the exact answer. This proves  $\hat{A}_k(q) \subseteq A_k(q)$  and rules out false positives. If an exact member is rejected, it satisfies the exact acceptance inequality but violates the represented one, namely

$$\hat{t}_k(o) < \kappa(q, o) \leq t_k(o).$$

Thus every false negative lies in the displayed boundary band, and objects outside that band have the same decision under both thresholds.

For the expansive case,  $\hat{t}_k(o) \geq t_k(o)$ . Every exact member then obeys

$$\kappa(q, o) \leq t_k(o) \leq \hat{t}_k(o),$$

so  $A_k(q) \subseteq \hat{A}_k(q)$  and verification cannot introduce false negatives. The only possible errors are objects in the opposite boundary band  $\{o : t_k(o) < \kappa(q, o) \leq \hat{t}_k(o)\}$ , which can be false positives.  $\square$

### A.4 Supplementary pooled-recall bound

The following calculation translates a relative error in squared Euclidean radius into a conservative recall statement under the same local model. It applies to the verifier component only; graph candidates that were never reached are outside this bound.

The bound in (29) follows directly from the same local mass model. Assume a restrictive  $L_2$  reconstruction and let

$$\epsilon_k = \max_o \frac{r_k(o)^2 - \hat{r}_k(o)^2}{r_k(o)^2} < 1.$$

Then, for every object,

$$\hat{r}_k(o)^2 \geq (1 - \epsilon_k) \cdot r_k(o)^2, \quad \hat{r}_k(o) \geq \sqrt{1 - \epsilon_k} r_k(o).$$

Under  $N_o(r) = \rho(o) \cdot V_{d(o)} \cdot r^{d(o)}$ , the fraction of the exact local ball retained by the represented ball is at least

$$\begin{aligned} \frac{N_o(\hat{r}_k(o))}{N_o(r_k(o))} &= \left(\frac{\hat{r}_k(o)}{r_k(o)}\right)^{d(o)} \\ &\geq (1 - \epsilon_k)^{d(o)/2} \\ &\geq (1 - \epsilon_k)^{d_{\max}/2}, \end{aligned}$$

where  $d_{\max} = \max_o d(o)$  and  $0 < 1 - \epsilon_k < 1$ . Averaging these per-object retention ratios with the exact answer-set sizes as weights gives the same lower bound for pooled recall. This is a verifier-side bound under the local model; it does not account for candidates that the finite graph traversal fails to visit.

### A.5 Quantization width and threshold error

This section connects the residual codebook to the previous recall bound. The codebook spacing is a direct knob: a smaller spacing narrows the threshold boundary where floor decoding can reject an exact member, and therefore gives a stronger model-based recall bound.

The one-sided property of floor decoding can be related directly to the spacing of the residual codebook. Let the ordered entries of  $\mathcal{B}_k$  be  $b_{0,k} < \dots < b_{J_k-1,k}$ , and assume that the codebook covers

every residual encoded at rank  $k$ . Define its maximum adjacent spacing by

$$\Delta_k = \max_{0 \leq j < j_k - 1} (b_{j+1,k} - b_{j,k}).$$

For a residual in the interval  $[b_{j,k}, b_{j+1,k})$ , floor decoding selects  $b_{j,k}$ , so the following bound holds:

$$0 \leq e_{o,k} - \tilde{e}_{o,k} < \Delta_k, \quad 0 \leq t_k(o) - \hat{t}_k(o) < \Delta_k. \quad (34)$$

The second inequality follows by subtracting the shared mean and basis terms in (21) and (22). Residuals equal to the largest codebook endpoint have zero error; including that endpoint handles the boundary case omitted by the open interval above.

For positive Euclidean thresholds, define  $r_{k,\min}^2 = \min_o r_k(o)^2$ . If  $\Delta_k < r_{k,\min}^2$ , then the restrictive relative error in (29) obeys

$$\epsilon_k \leq \frac{\Delta_k}{r_{k,\min}^2},$$

and the local ball-mass model gives the conservative pooled bound

$$\text{Recall}_{\text{pool}} \geq \left(1 - \frac{\Delta_k}{r_{k,\min}^2}\right)^{d_{\max}/2}. \quad (35)$$

More locally, an object with radius  $r_k(o)$  retains at least the fraction  $(1 - \Delta_k/r_k(o)^2)^{d(o)/2}$  of its modeled ball. These bounds explain the role of residual width without asserting that a finite dataset exactly follows the local model; the measured verifier curves provide the empirical check.

## A.6 Candidate coverage and verifier retention

The next identity states the two-part interpretation of end-to-end recall. A query can lose an exact answer because the graph never returns it, or because the verifier rejects a returned candidate. The factors below name those two events without assuming that they are statistically independent.

The two error sources in the indexed query admit an exact recall decomposition. For a query with nonempty exact answer  $A = A_k(q)$ , let  $C = C(q)$  be the graph candidate set and let  $\hat{R} = \hat{R}_k(q) \subseteq C$  be the verified output. Then

$$\begin{aligned} \text{Recall}(q) &= \frac{|A \cap \hat{R}|}{|A|} \\ &= \underbrace{\frac{|A \cap C|}{|A|}}_{\text{candidate coverage}} \underbrace{\frac{|A \cap \hat{R}|}{|A \cap C|}}_{\text{verifier retention}}, \end{aligned} \quad (36)$$

where the second factor is defined as zero when  $|A \cap C| = 0$ . This identity is purely set-theoretic. The first factor is determined by the lifted graph traversal; the second is determined by the represented threshold on the candidates that were actually reached. Under exact thresholds, verifier retention is one, so recall is limited only by graph coverage. Under a restrictive floor-decoded threshold, candidate coverage is unchanged while verifier retention can drop only for the boundary band in Proposition 4.

This factorization also explains the experimental separation: the lift and graph ablations target the first factor, while the M-only and verifier-capacity controls target the second. It is not a claim that the two factors are statistically independent; it is an exact decomposition for each query.

## A.7 Conditional completeness beyond the construction pivot

Finally, we make the scope of the cross-pivot argument explicit. The score correction is exact for a represented target rank, but exact end-to-end recovery additionally requires the graph traversal to cover the corresponding target envelope. The next proposition states that implication under that explicit condition.

The target-rank correction gives a useful conditional statement for cross-pivot queries. For the  $L_2$  path, let  $E_k(q)$  denote the target envelope evaluated by the corrected score in (30). In the following proposition, “exhaustive” refers to the graph traversal, not merely to the verifier scan.

**PROPOSITION 5 (CONDITIONAL CROSS-PIVOT COMPLETENESS).** *Assume exact thresholds and exact arithmetic. If the traversal for  $(q, k)$  is exhaustive over  $E_k(q)$  and reaches every indexed object in that envelope, then the verifier returns the exact target- $k$  answer, including when  $k > K_{\text{pivot}}$ .*

**PROOF OF PROPOSITION 5.** By Lemma 1, the corrected construction-rank score is exactly  $s_{t_k}(q, o)$  when the target threshold is represented without error. Equation (10) therefore gives

$$E_k(q) = \{o : s_{t_k}(q, o) \geq \frac{1}{2} \cdot \|q\|^2\} = A_k(q).$$

The exhaustive-traversal assumption supplies every member of  $A_k(q)$  to the verifier. Because the verifier uses the exact same threshold predicate, it accepts every such member and rejects every non-member in the visited envelope. Hence its output is exactly  $A_k(q)$ . The proof does not assert that a finite-beam ANN traversal is exhaustive; that coverage condition is the empirical quantity tested by the cross-pivot experiments.  $\square$